OPERATING SYSTEM



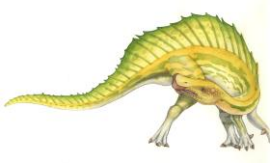
By Dr. Hadi Abdullah

Assistant Professor CSE



Instructor Contact Details

- Name: Dr. Hadi Abdullah
- Course Instructor: Operating System
- Bachelor in Computer engineering from COMSATs Abbottabad 2007-2011
- Masters in Electrical (Computer system) engineering from UTHM Malaysia 2012-2014
- PhD in Electrical (Computer system) engineering from UTHM Malaysia 2015-2019
- Multiple projects with companies
- More than 20 research articles, author in books, product patents and copyrights
- Multiple gold medals in international product competitions





Operating System

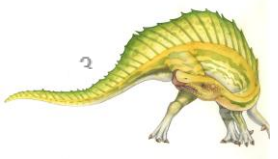
Reading Material

Textbook

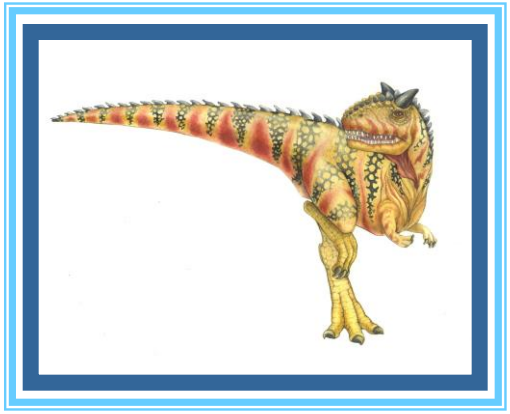
- Operating Systems Concepts by Silberschatz, Galvin, and Gagne, 10th Ed, 2018.

References Material:

1. Andrew S Tanenbaum and Herbert Bos. Modern Operating Systems. 4th Ed, 2014.
2. Operating Systems Principles & Practice by Thomas Anderson and Michael Dahlin 2nd Ed, 2014.
3. Kay Robbins & Steve Robbins. Unix Systems Programming, 2nd Ed, 2015.



Chapter 1: Introduction



Silberschatz, Galvin and Gagne ©2018



Chapter 1: Introduction

- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Operations
- Resource Management
- Security and Protection
- Virtualization
- Distributed Systems
- Kernel Data Structures
- Computing Environments
- Free/Libre and Open-Source Operating Systems





Objectives

- Describe the general organization of a computer system and the role of interrupts
- Describe the components in a modern, multiprocessor computer system
- Illustrate the transition from user mode to kernel mode
- Discuss how operating systems are used in various computing environments
- Provide examples of free and open-source operating systems





What Does the Term Operating System Mean?

- An operating system is “fill in the blanks”
- What about:
  - Car
  - Airplane
  - Printer
  - Washing Machine
  - Toaster
  - Compiler
  - Etc.





What Does the Term Operating System Mean?

What Does the Term Operating System Mean?

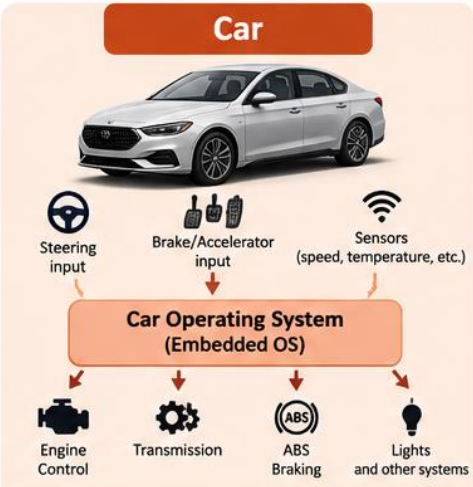
An operating system is the **software** that **manages hardware resources** and provides services to run applications (programs) — not just in computers, but in many devices around us!

Computer



Manages hardware and runs applications

Car



Manages car hardware and control systems

Airplane



Manages airplane systems reliably and safely

Printer



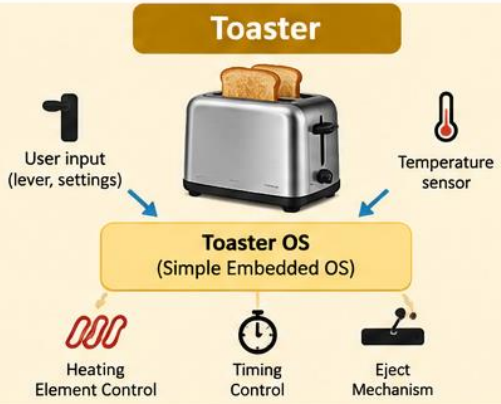
Manages printer hardware and executes print jobs

Washing Machine



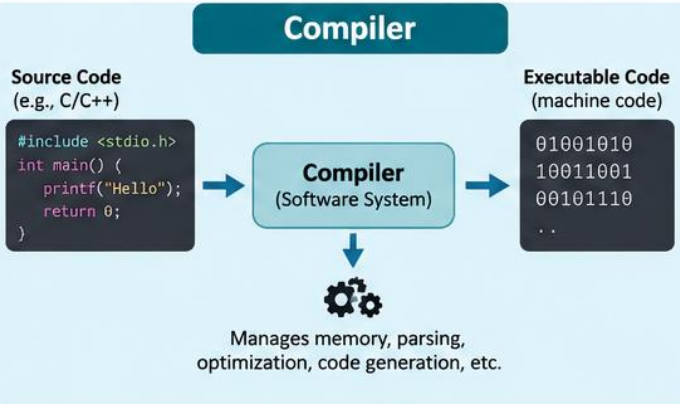
Manages washing machine functions automatically

Toaster



Manages toasting time and temperature

Compiler



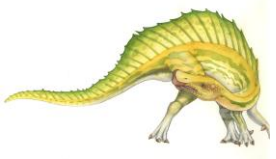
An operating system for the **compilation process**

The concept of an Operating System applies to many devices — any system software that manages hardware and provides services!



What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner



Computer System Structure

Four components working together to solve real-world problems

USERS

Users: people, machines, other computers



Human user



Other computers (networked devices)



Smart machines (e.g., robots, IoT devices)

Send requests (run programs)

Results and services (answers, output)

APPLICATION PROGRAMS

Applications use system resources to solve user problems



Word processor



Compiler (development tool)



Web browser



Database system



Video game

System calls (request resources)

Services (provide results)

OPERATING SYSTEM

Controls and coordinates hardware use among applications and users



Resource management



CPU scheduling



Memory management



File system management



Device control (I/O management)



Security and protection



OS acts as the **bridge** between software **and hardware.**

Controls hardware (issues commands)

Status and data (interrupts, device responses)

HARDWARE

Provides basic computing resources: CPU, memory, I/O devices



CPU (processor)



Memory (RAM)



Storage (HDD/SSD)



Input devices (keyboard, mouse)



Output devices (monitor)



Other I/O devices (printer, etc.)

...

Together, these four components form a complete computer system that lets users run applications on hardware.

What Operating Systems Do

1 Personal Users
(PCs & Laptops)



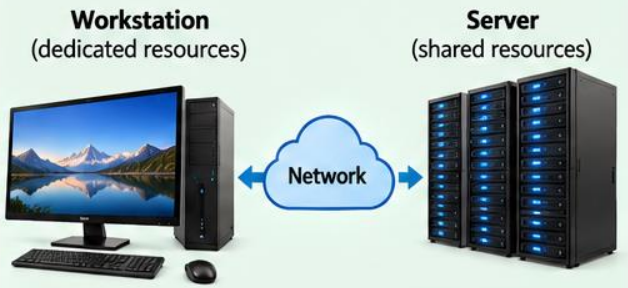
- Convenience
- Ease of use
- Good Performance

2 Shared Computers
(Mainframes / Minicomputers)



- Multiple Users
- Fair Resource Allocation

3 Workstations and Servers



Operating System

- Resource Allocation
- Control
- Manage Programs
- Provide Services

4 Mobile Devices
(Smartphones & Tablets)



- Battery Life
- Touch Interface
- Voice Recognition

Hardware



CPU · Memory · Storage · I/O Devices

5 Embedded Systems
(in Devices and Automobiles)



Little or No User Interface

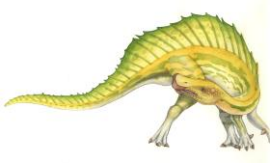
Different Users | **Different Systems** | **Same Core Role** | **Make Computing Useful**





What Operating Systems Do

- Depends on the point of view
- Users want convenience, **ease of use** and **good performance**
  - Don't care about **resource utilization**
- But shared computer such as **mainframe** or **minicomputer** must keep all users happy
  - Operating system is a **resource allocator** and **control program** making efficient use of HW and managing execution of user programs
- Users of dedicate systems such as **workstations** have dedicated resources but frequently use shared resources from **servers**
- Mobile devices like smartphones and tables are resource poor, optimized for usability and battery life
  - Mobile user interfaces such as touch screens, voice recognition
- Some computers have little or no user interface, such as embedded computers in devices and automobiles
  - Run primarily without user intervention





Defining Operating Systems

- Term OS covers many roles
  - Because of myriad designs and uses of OSes
  - Present in toasters through ships, spacecraft, game machines, TVs and industrial control systems
  - Born when fixed use computers for military became more general purpose and needed resource management and program control



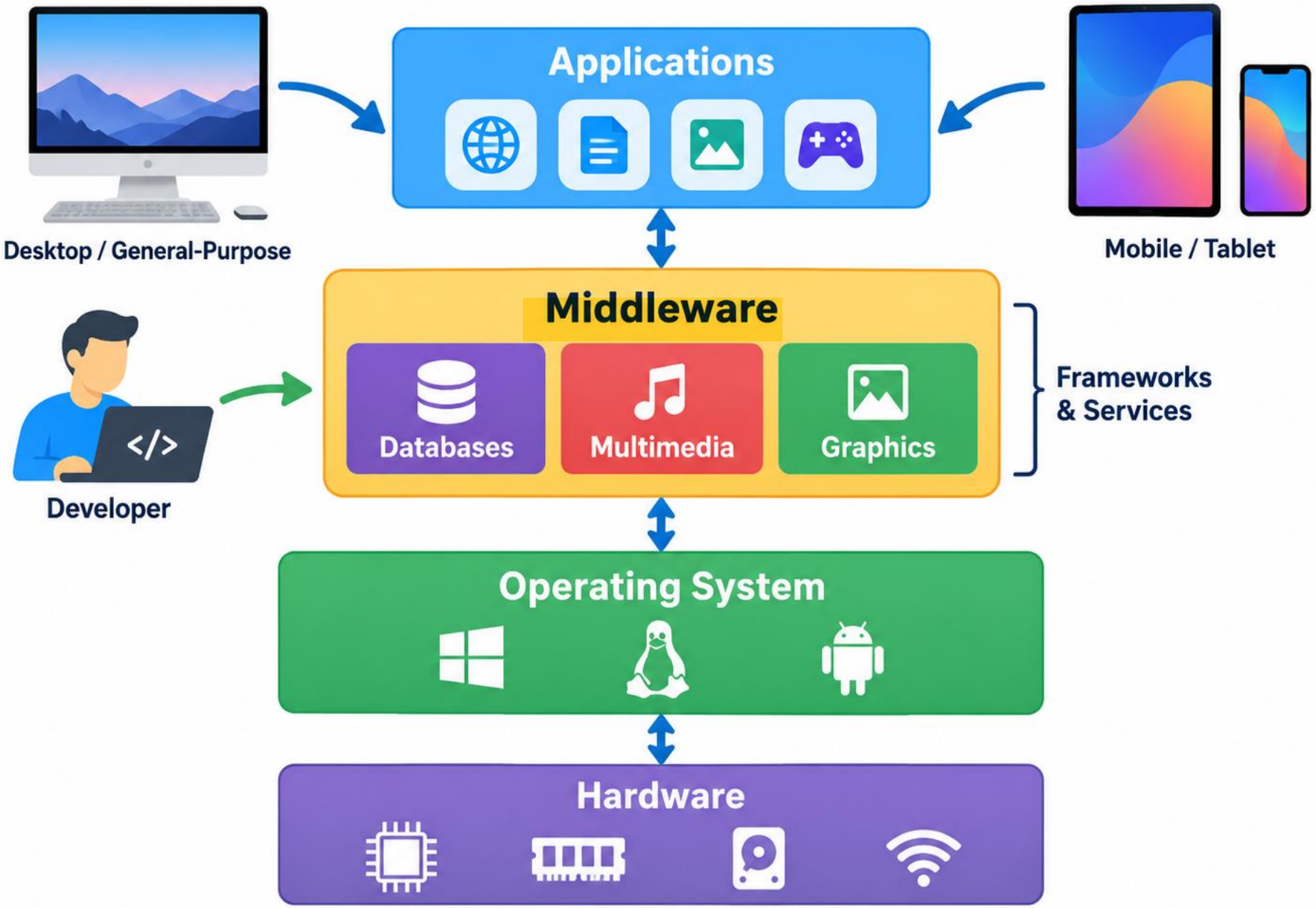


Operating System Definition

- No universally accepted definition
- "Everything a vendor ships when you order an operating system" is a good approximation
  - But varies wildly
- "The one program running at all times on the computer" is the **kernel,** part of the operating system
- Everything else is either
  - A ***system program*** (ships with the operating system, but not part of the kernel) , or
  - An ***application program***, all programs not associated with the operating system
- Today's OSes for general purpose and mobile computing also include ***middleware*** – a set of software frameworks that provide addition services to application developers such as databases, multimedia, graphics

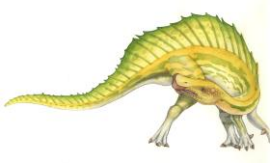


Middleware in Modern Operating Systems





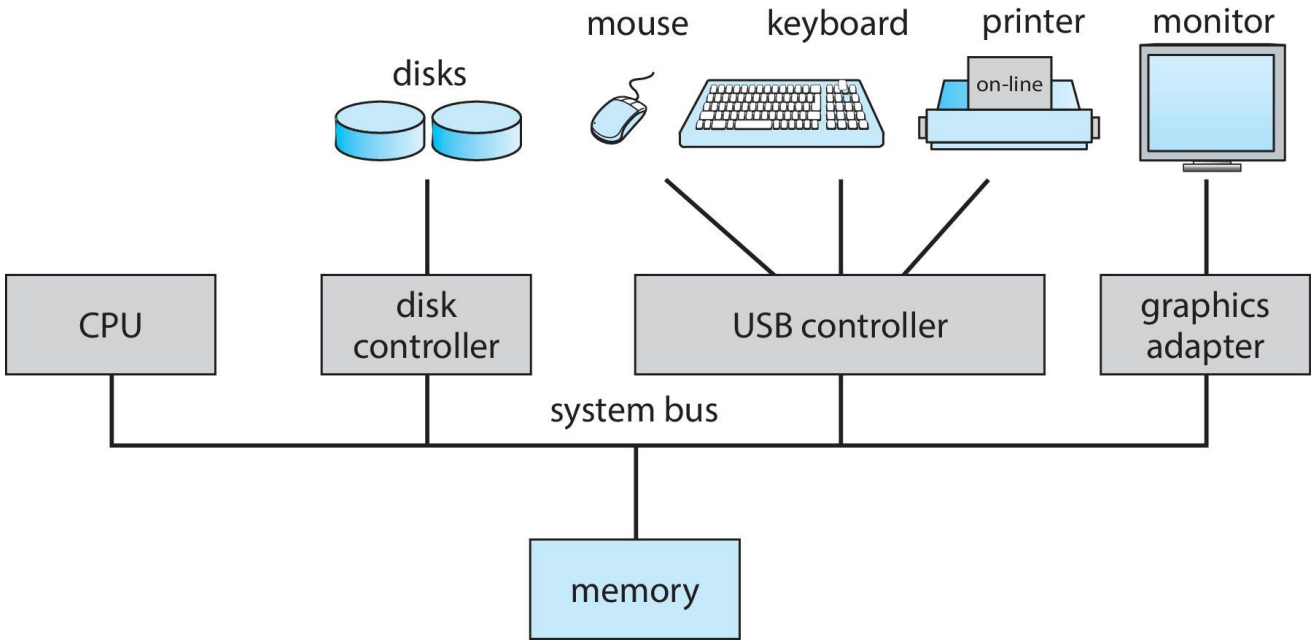
Overview of Computer System Structure

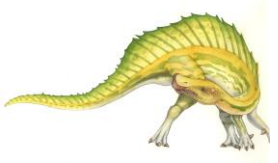




Computer System Organization

- Computer-system operation
  - One or more CPUs, device controllers connect through common **bus** providing access to shared memory
  - Concurrent execution of CPUs and devices competing for memory cycles





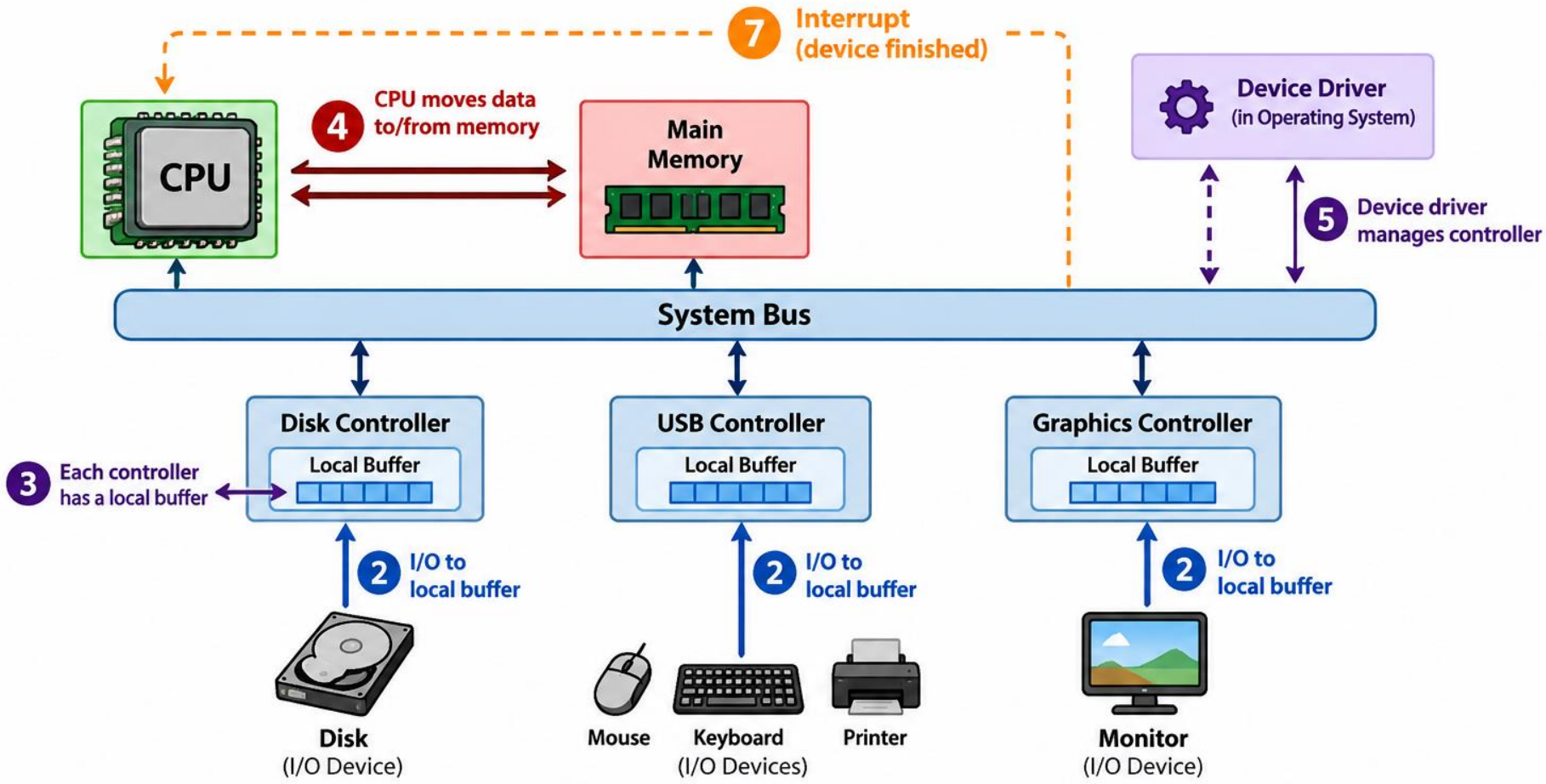


Computer-System Operation

- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- Each device controller type has an operating system **device driver** to manage it
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller
- Device controller informs CPU that it has finished its operation by causing an **interrupt**



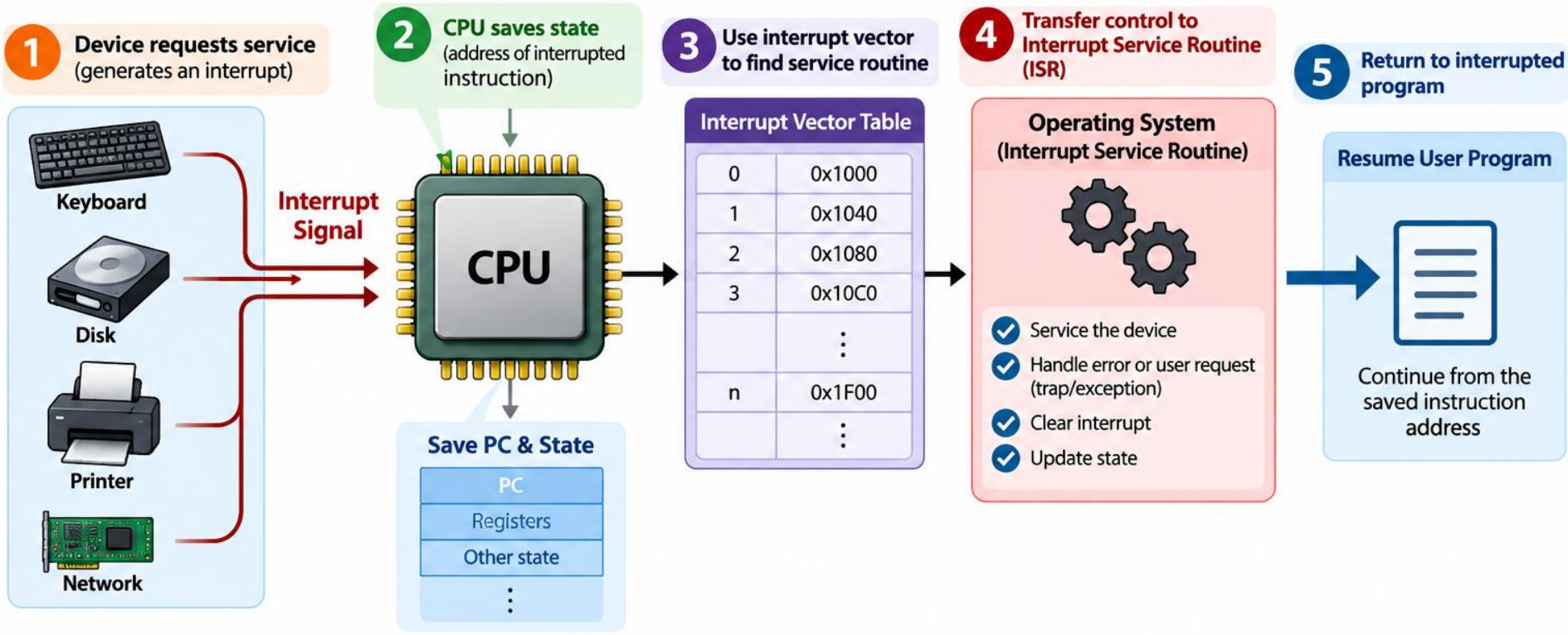
Computer-System Operation







Common Functions of Interrupts



Types of Interrupts

Hardware Interrupt

(from I/O devices)

Trap / Exception

(software-generated by error or user request)

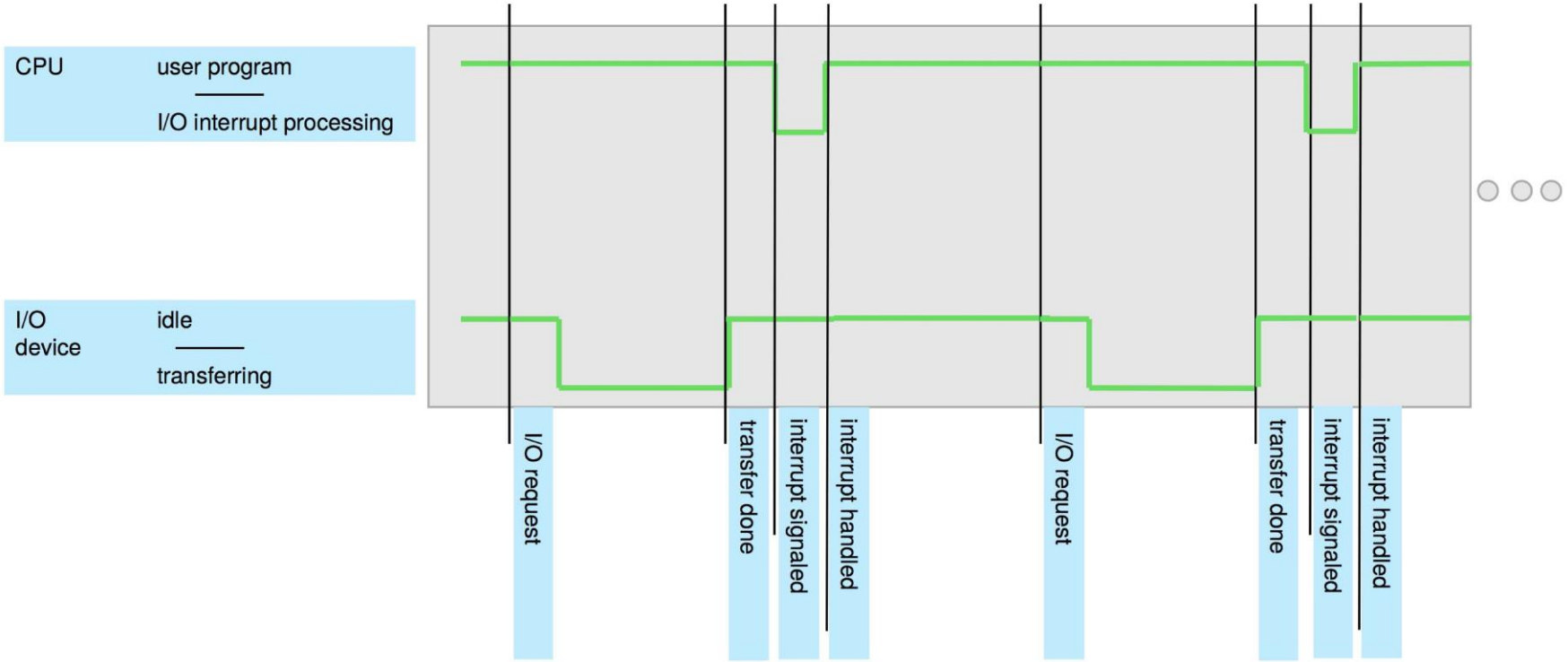
Key Takeaway

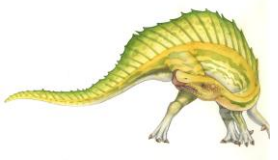
An operating system is **interrupt driven**





Interrupt Timeline







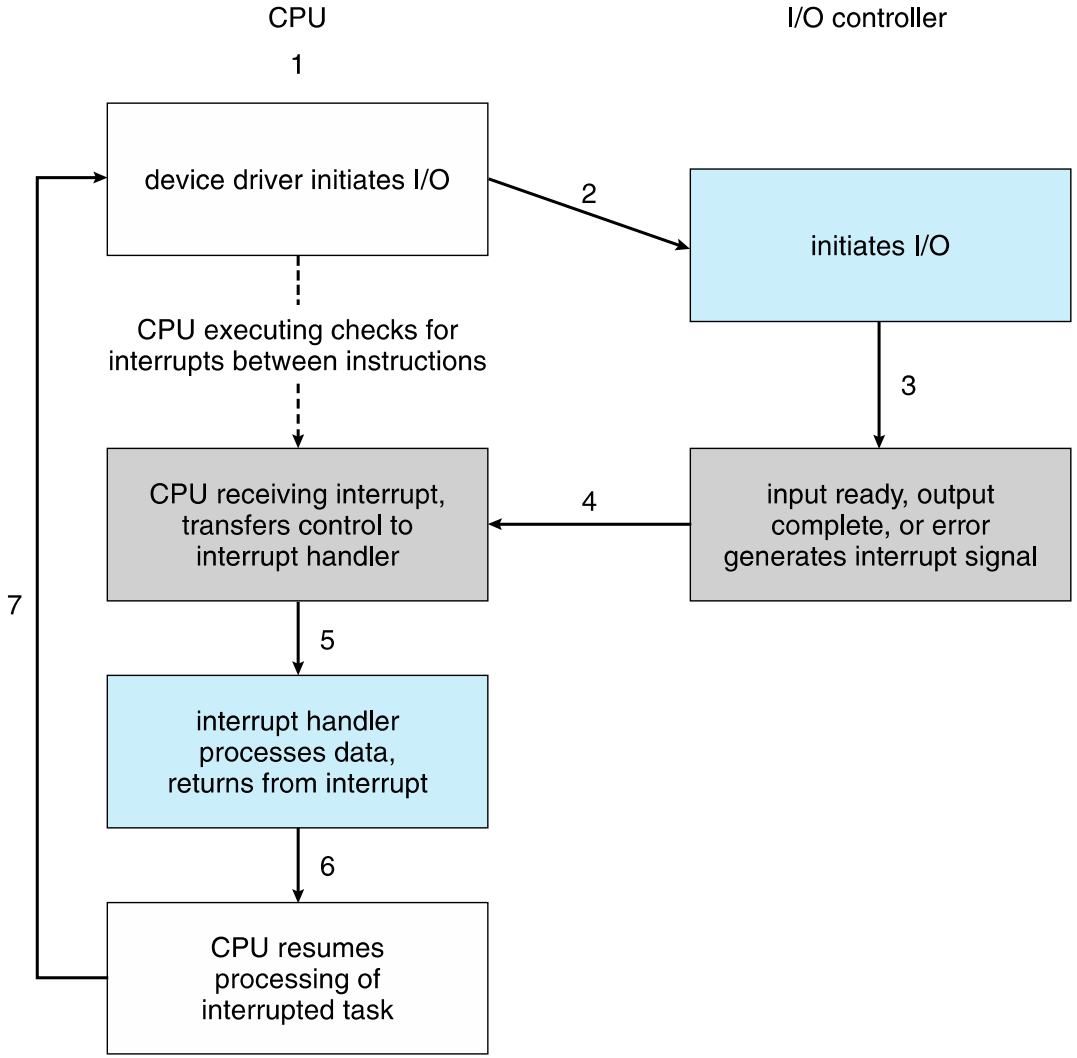
Interrupt Handling

- The operating system preserves the state of the CPU by storing the registers and the program counter
- Determines which type of interrupt has occurred:
- Separate segments of code determine what action should be taken for each type of interrupt





Interrupt-drive I/O Cycle

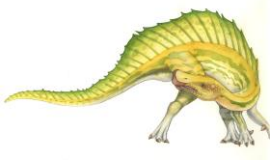






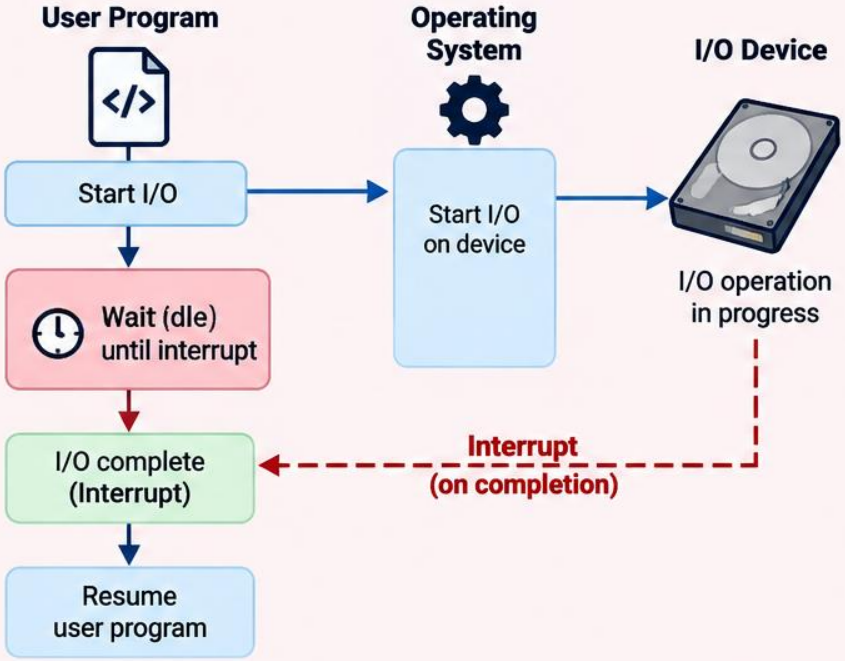
I/O Structure

- Two methods for handling I/O
  - After I/O starts, control returns to user program only upon I/O completion
  - After I/O starts, control returns to user program without waiting for I/O completion

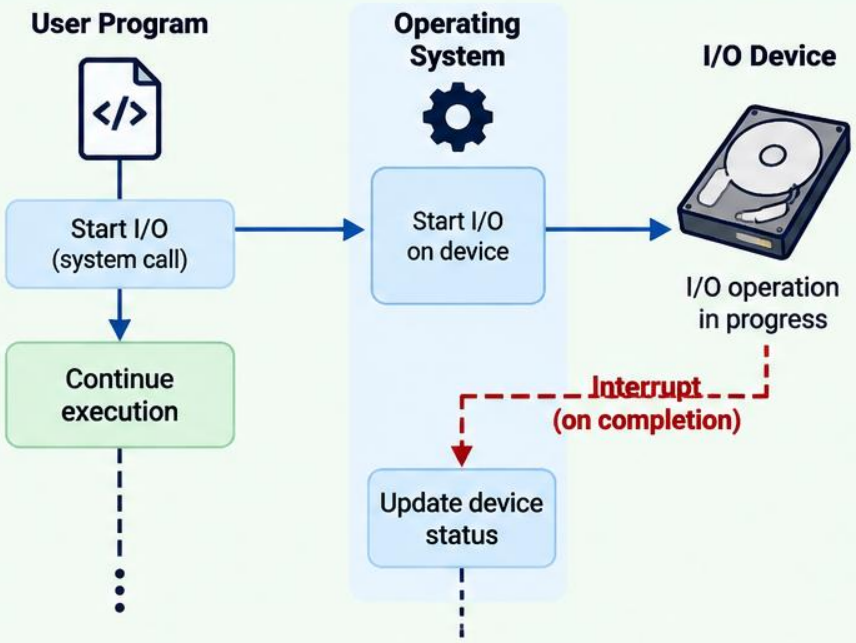


I/O Structure

1 Blocking I/O (Wait for completion)



2 Non-blocking I/O (Continue without waiting)



3 Device-Status Table

Device	Type	Address	Status
Disk	Block device	0x1F0	Idle
Printer	Character device	0x378	Busy
Network	Network device	0xC000	Idle
...	...	...	...

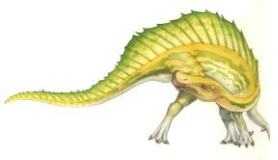
At most one I/O request outstanding per device





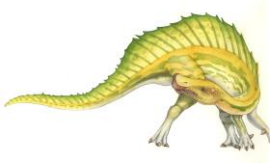
I/O Structure

- After I/O starts, control returns to user program only upon I/O completion
  - Wait instruction idles the CPU until the next interrupt
  - Wait loop (contention for memory access)
  - At most one I/O request is outstanding at a time, no simultaneous I/O processing
- After I/O starts, control returns to user program without waiting for I/O completion
  - **System call** – request to the OS to allow user to wait for I/O completion
  - **Device-status table** contains entry for each I/O device indicating its type, address, and state
  - OS indexes into I/O device table to determine device status and to modify table entry to include interrupt





Storage Structure





Storage Structure

- Main memory – only large storage media that the CPU can access directly
  - **Random access**
  - Typically **volatile**
  - Typically **random-access memory** in the form of **Dynamic Random-access Memory (DRAM)**
- Secondary storage – extension of main memory that provides large **nonvolatile** storage capacity





Storage Structure (Cont.)

- **Hard Disk Drives** (**HDD**) – rigid metal or glass platters covered with magnetic recording material
  - Disk surface is logically divided into **tracks**, which are subdivided into **sectors**
  - The **disk controller** determines the logical interaction between the device and the computer
- **Non-volatile memory** (**NVM**) devices– faster than hard disks, nonvolatile
  - Various technologies
  - Becoming more popular as capacity and performance increases, price drops





Storage Definitions and Notation Review

Bit

Basic unit of storage
Value: 0 or 1

0 1

Byte

8 bits
Smallest convenient unit of storage

0 1 0 1 0 1 0 1

1 byte = 8 bits

Word

Architecture's native unit of data
(e.g., 64-bit = 8 bytes)

1 word = 8 bytes (64 bits)

Common Storage Units

Unit	Abbreviation	Size (in bytes)
Kilobyte	KB	1,024 (2^{10})
Megabyte	MB	$1{,}024^2$ (2^{20})
Gigabyte	GB	$1{,}024^3$ (2^{30})
Terabyte	TB	$1{,}024^4$ (2^{40})
Petabyte	PB	$1{,}024^5$ (2^{50})

Key Points

- All computer storage is based on **bits**.
- A bit has a value 0 or 1.
- A byte is 8 bits.
- A word is one or more bytes (e.g., 64-bit = 8 bytes).
- Storage and most throughput are measured in bytes and multiples (KB, MB, GB, TB, PB).
- Networking measurements are usually in bits (exception).

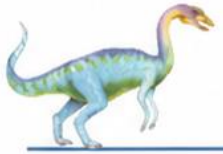




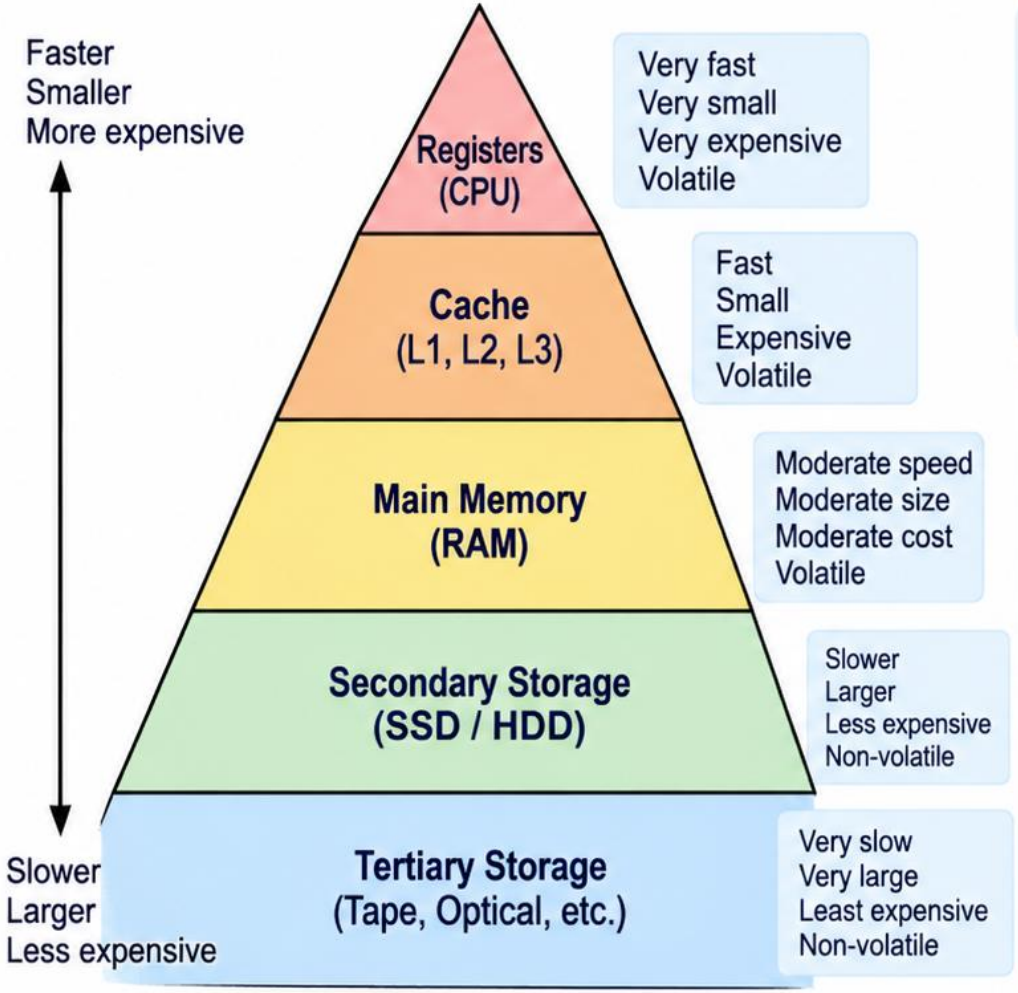
Storage Hierarchy

- Storage systems organized in hierarchy
  - Speed
  - Cost
  - Volatility
- **Caching** – copying information into faster storage system; main memory can be viewed as a cache for secondary storage
- **Device Driver** for each device controller to manage I/O
  - Provides uniform interface between controller and kernel





Storage Hierarchy



Storage systems organized in hierarchy

- Storage systems organized in hierarchy
  - Speed
  - Cost
  - Volatility

Caching

- Copying information into a faster storage system; main memory can be viewed as a cache for secondary storage.

Device Driver

- For each device controller to manage I/O
  - Provides uniform interface between controller and kernel.

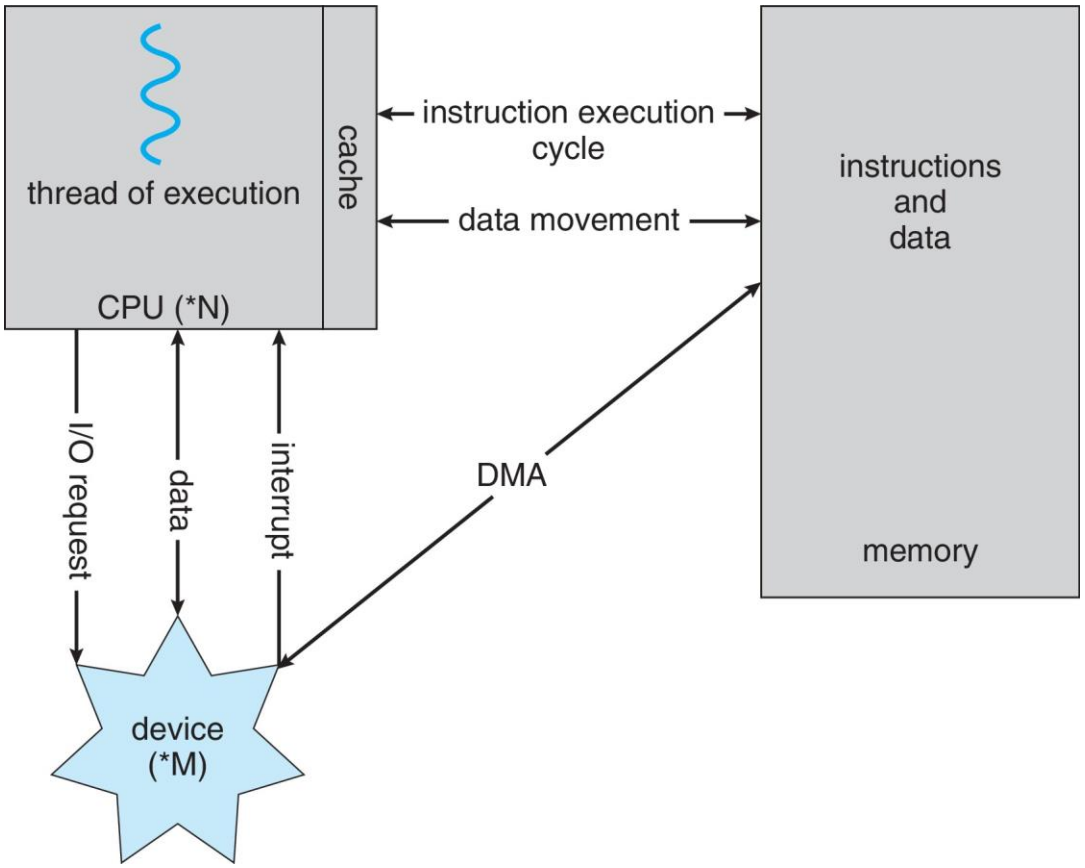




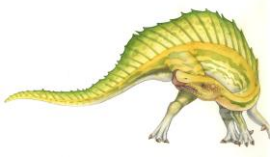
How a Modern Computer Works

After setting up buffers, pointers, and counters for the I/O device, the device controller transfers an entire block of data directly to or from the device and main memory, with no intervention by the CPU. Only one interrupt is generated per block, to tell the device driver that the operation has completed, rather than the one interrupt per byte generated for low-speed devices. While the device controller is performing these operations, the CPU is available to accomplish other work.

Direct memory access (DMA)



A von Neumann architecture

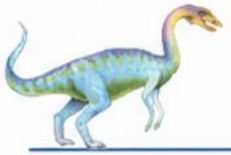




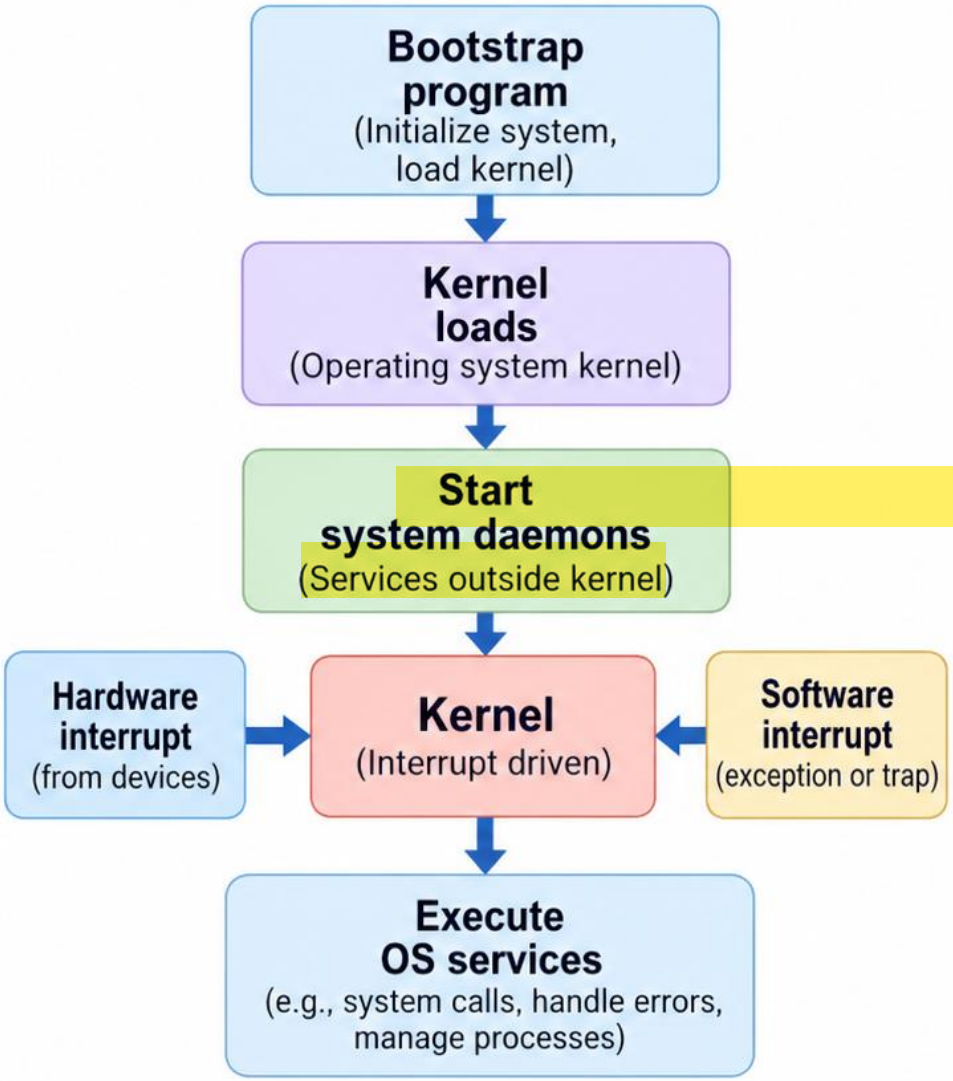
Direct Memory Access Structure

- Used for high-speed I/O devices able to transmit information at close to memory speeds
- Device controller transfers blocks of data from buffer storage directly to main memory without CPU intervention
- Only one interrupt is generated per block, rather than the one interrupt per byte





Operating-System Operations

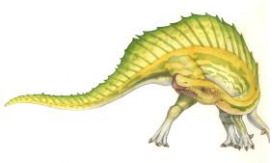


- **Bootstrap program** – simple code to initialize the system, load the kernel
- **Kernel loads**
- Starts **system daemons** (services provided outside of the kernel)
- **Kernel interrupt driven** (hardware and software)
  - Hardware interrupt by one of the devices
  - Software interrupt (**exception** or **trap**):
    - Software error (e.g., division by zero)
    - Request for operating system service – **system call**
    - Other process problems include infinite loop, processes modifying each other or the operating system





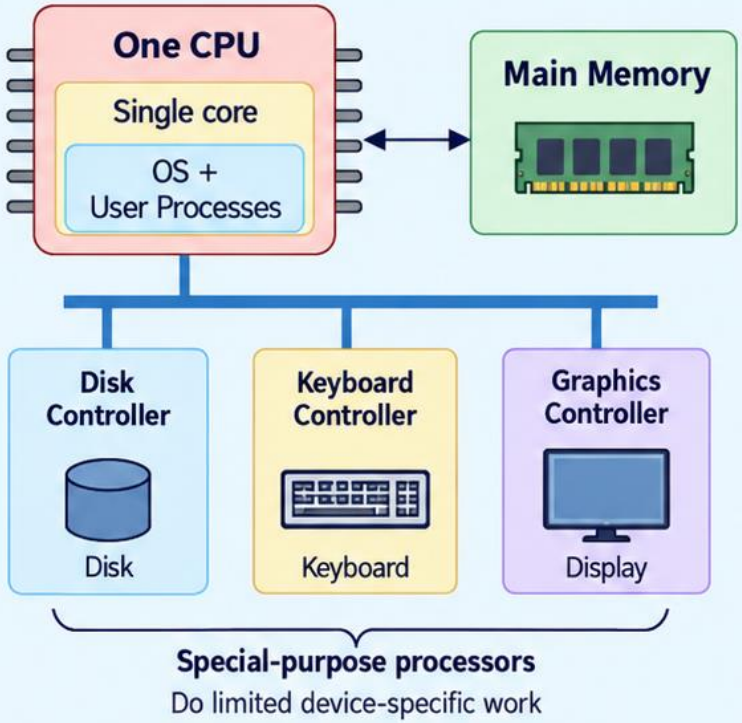
Computer-System Architecture





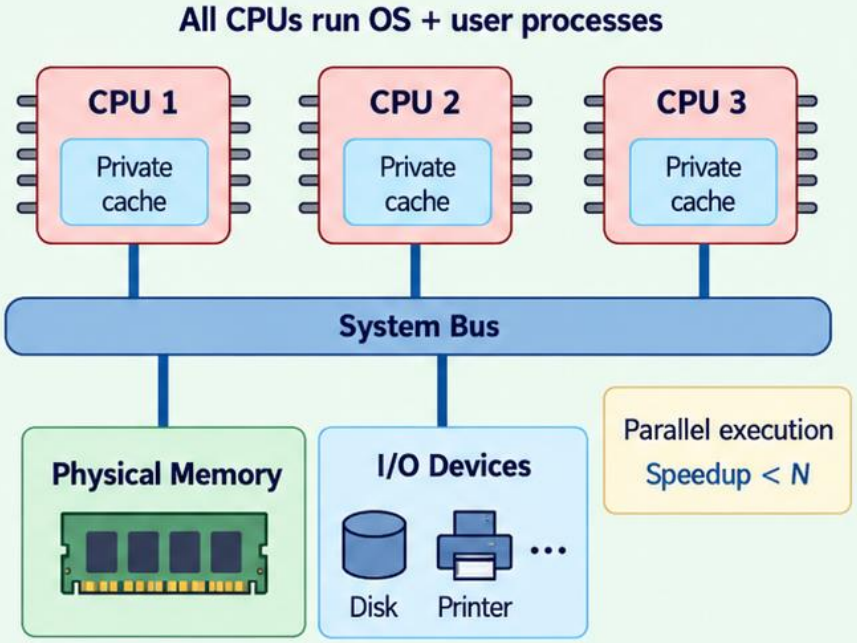
Single-Processor vs Multiprocessor Systems

Single-Processor System



- One general-purpose CPU
- Device-specific controllers
- Not multiprocessor

Multiprocessor System (SMP)



- Two or more CPUs
- Shared memory & devices
- Higher throughput
- SMP: peer CPUs

Single Processor = 1 general-purpose CPU → **Multiprocessor = multiple peer CPUs**



Multicore Systems

Why Multicore?

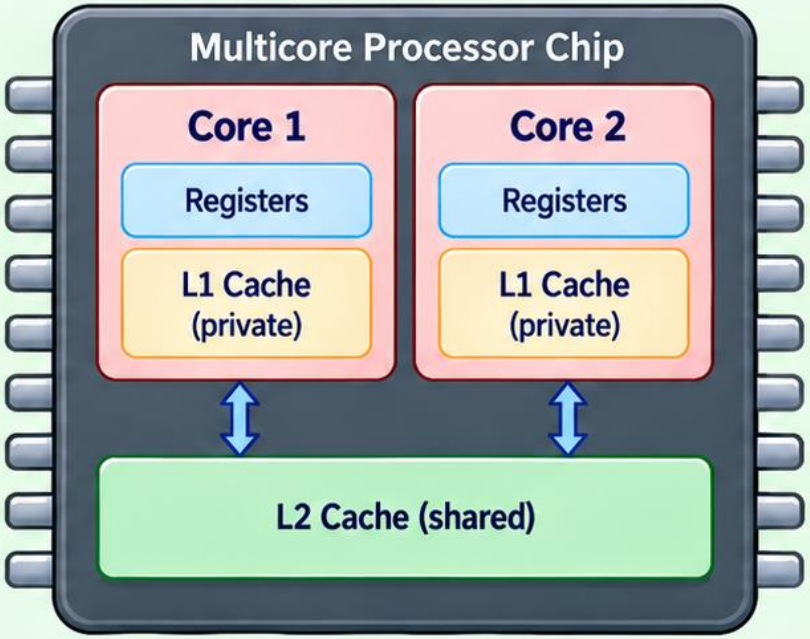
Faster on-chip communication
Cores on the same chip can communicate much faster than separate chips.

Lower power consumption
One chip with multiple cores uses less power than multiple single-core chips.

Better performance with parallel work
Multiple cores can run multiple threads or processes at the same time.

Modern multiprocessors often take the form of multicore chips.

Dual-Core Chip Design



Local caches are smaller and faster; shared caches are larger and higher-level.

How the OS Sees It

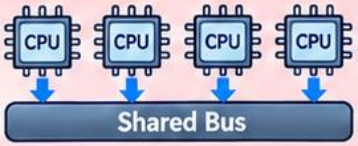
Core 1 + Core 2 + ··· + Core N → **OS sees N CPUs**

- OS schedules work across cores
- Applications and OS must use cores efficiently
- Supported by Windows, macOS, Linux, Android, and iOS

Multicore processor = multiple cores on one chip, seen by the OS as multiple CPUs

NUMA Systems

Why SMP Struggles to Scale



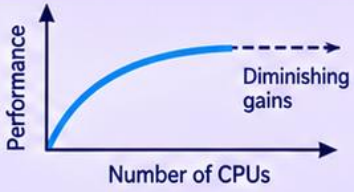
More CPUs increase contention

As processors are added, they compete for the shared system bus.



Bus becomes a bottleneck

Too many CPUs can overload the interconnect and reduce efficiency.

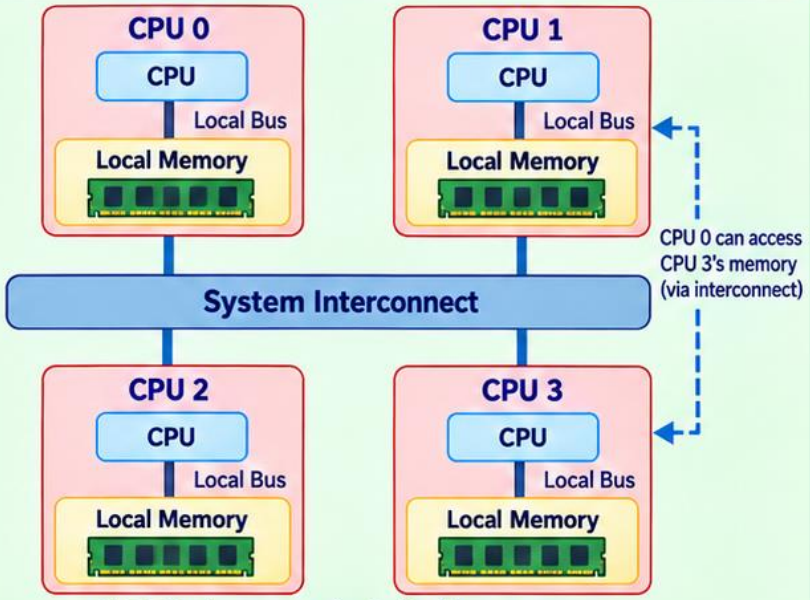


Performance stops scaling well

Computing power rises at first, then gains shrink as contention grows.

Traditional shared-bus multiprocessors do not scale indefinitely.

NUMA Architecture



All CPUs share one physical address space.

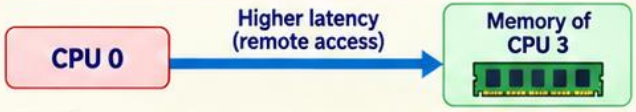
Each CPU (or CPU group) has its own local memory, accessed quickly over a small local bus.

Benefits and Trade-Offs

Advantages

- Fast local memory access
- Little or no contention on local access
- Scales better as processors are added

Potential Drawback



- Remote memory access has higher latency
- CPU0 reaches CPU3's memory more slowly than its own

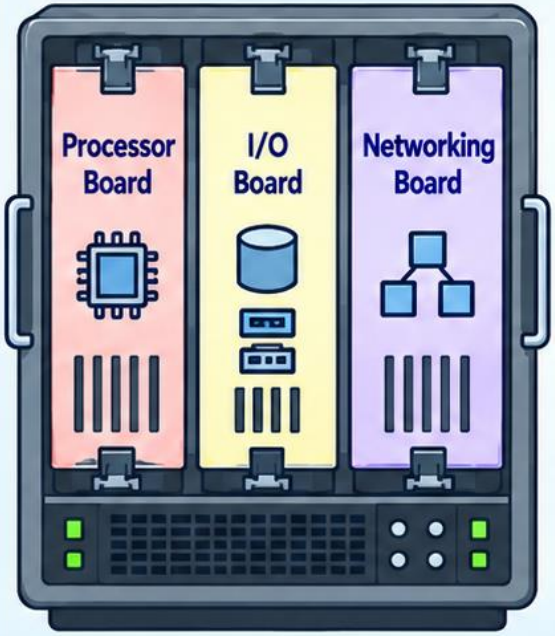
OS Support

- CPU scheduling and memory management reduce NUMA penalties
- Popular in servers and high-performance computing

NUMA = processors with local memory, connected by a shared interconnect for a common address space

Blade Servers

Inside a Blade Chassis



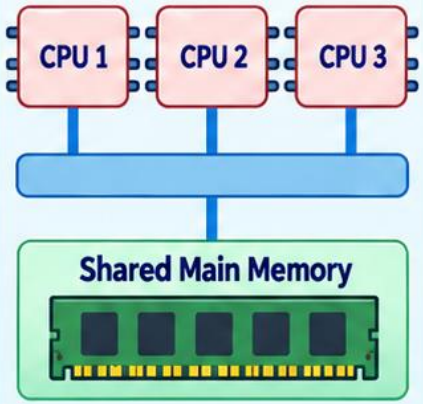
Shared chassis
Power, cooling and management are shared.

Multiple boards
Processor, I/O and networking boards in one chassis.

Modular design
Add or replace boards easily.

Traditional Multiprocessor vs Blade Server

Traditional multiprocessor



One system, one OS

Blade server



Each blade boots independently

Key Characteristics

- High density in a compact chassis
- Independent blades (each runs its own OS)
- Some blades are multiprocessor systems



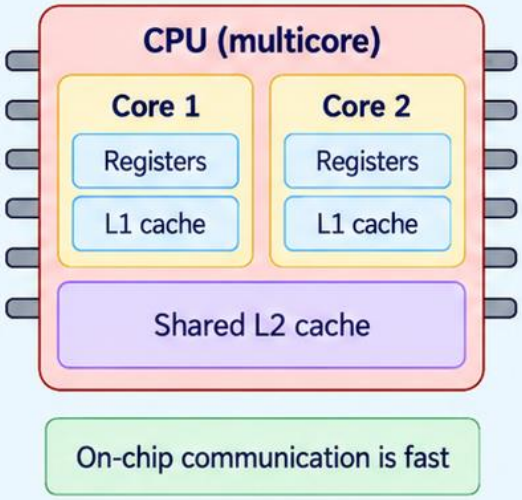
More computing power in less space.

Blade server = multiple independent boards in one chassis, each running its own operating system.



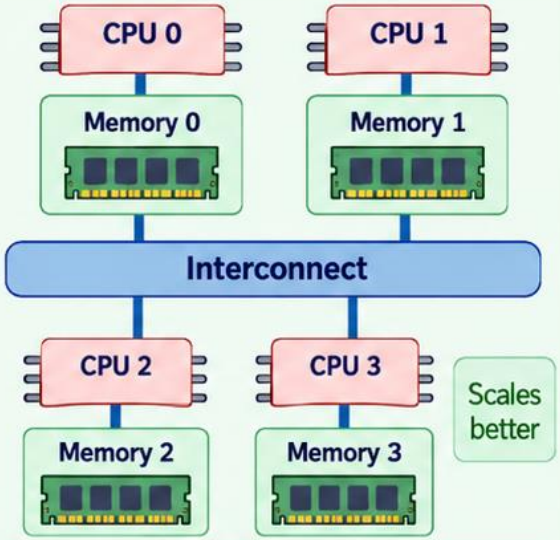
Multicore, NUMA, and Blade Servers

Multicore System



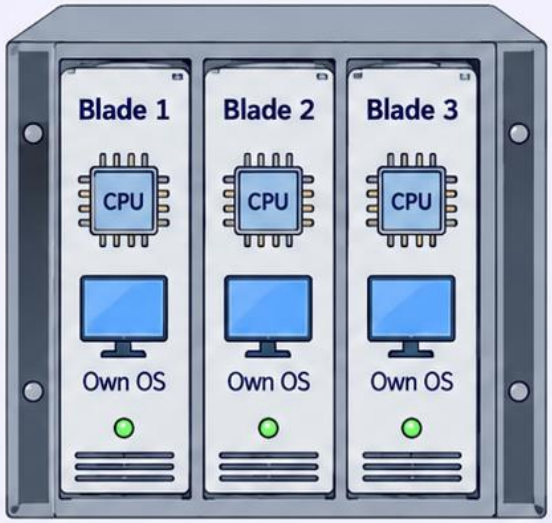
- Multiple cores on one chip
- OS sees cores like CPUs
- Faster & lower power

NUMA Architecture



- Each CPU has local memory
- Fast local access
- Remote memory = higher latency

Blade Servers



- Multiple boards in one chassis
- Each blade boots independently
- Like multiple separate systems

Quick Definitions

CPU	Processor	Core	Multicore	Multiprocessor
Executes instructions	Chip with one or more CPUs	Basic computation unit	Multiple cores on one CPU	Multiple processors

Modern systems use multicore CPUs; large servers may use NUMA or blade designs.





Clustered Systems

What is a clustered system?

- A group of two or more individual computers (**nodes**) connected via a network (e.g., LAN or high-speed interconnect).
- Each node is typically a multicore system.
- Nodes share storage and work together, forming a **loosely coupled** system.

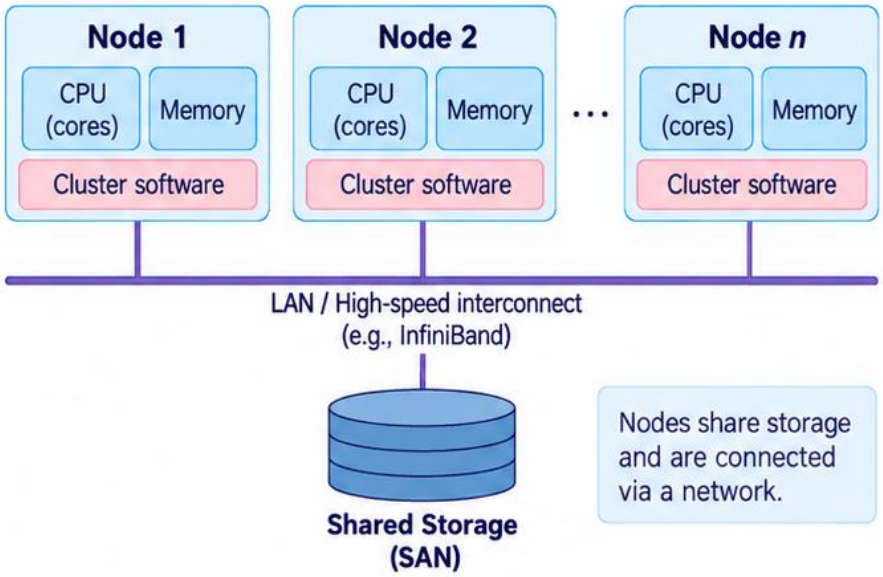
Main purposes

- **High availability** – continue service even if one or more nodes fail.
- **Graceful degradation** – performance continues proportional to surviving hardware.
- **Fault tolerance** – can continue operation even after a component failure (with detection, diagnosis, and recovery).
- **High-performance computing** – run applications in parallel across many nodes.

Cluster types

- **Asymmetric clustering** – one active node and one hot-standby node (standby monitors the active node).
- **Symmetric clustering** – two or more nodes actively running applications and monitoring each other.
- **Parallel clusters** – multiple hosts access shared data (e.g., databases), often requiring special software and a distributed lock manager (DLM).

General Structure of a Clustered System



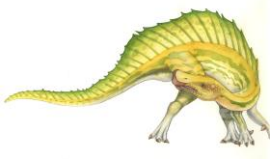
Key Takeaways

- Clustered systems provide high availability, reliability, and scalability.
- Nodes are loosely coupled and communicate over a network.
- Clusters can also be used for high-performance computing (using parallelization).
- With shared storage (e.g., SAN), any node can take over applications if another node fails.





Operating-System Operations





From Power-On to a Running Operating System

The Boot Process

A computer starts by running a simple bootstrap program in firmware, which initializes the hardware, loads the operating system kernel, and starts system programs (daemons).

1 Power On / Reboot



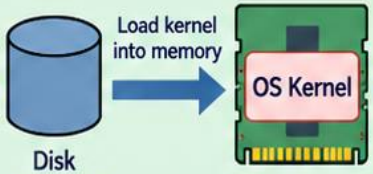
System is powered up or rebooted

2 Run Bootstrap Program



- Stored in hardware (firmware)
- Initializes CPU, devices, memory
- Knows how to locate and load the OS kernel

3 Load OS Kernel



4 Start System Programs (Daemons)

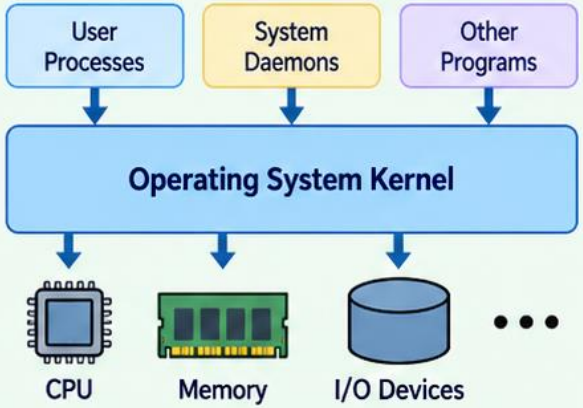
- systemd (first program on Linux)
- Other system daemons
- Device managers, services, etc.

5 System Ready



System is fully booted and waits for events (interrupts)

System State After Boot



Types of Interrupts

Hardware Interrupt



- Signaled by an I/O device or other hardware event
- Notifies the OS that something needs service

Trap (Exception)



- Software-generated interrupt
- Caused by an error (e.g., division by zero, invalid memory access)
- Or a system call request from a user program

Key Points

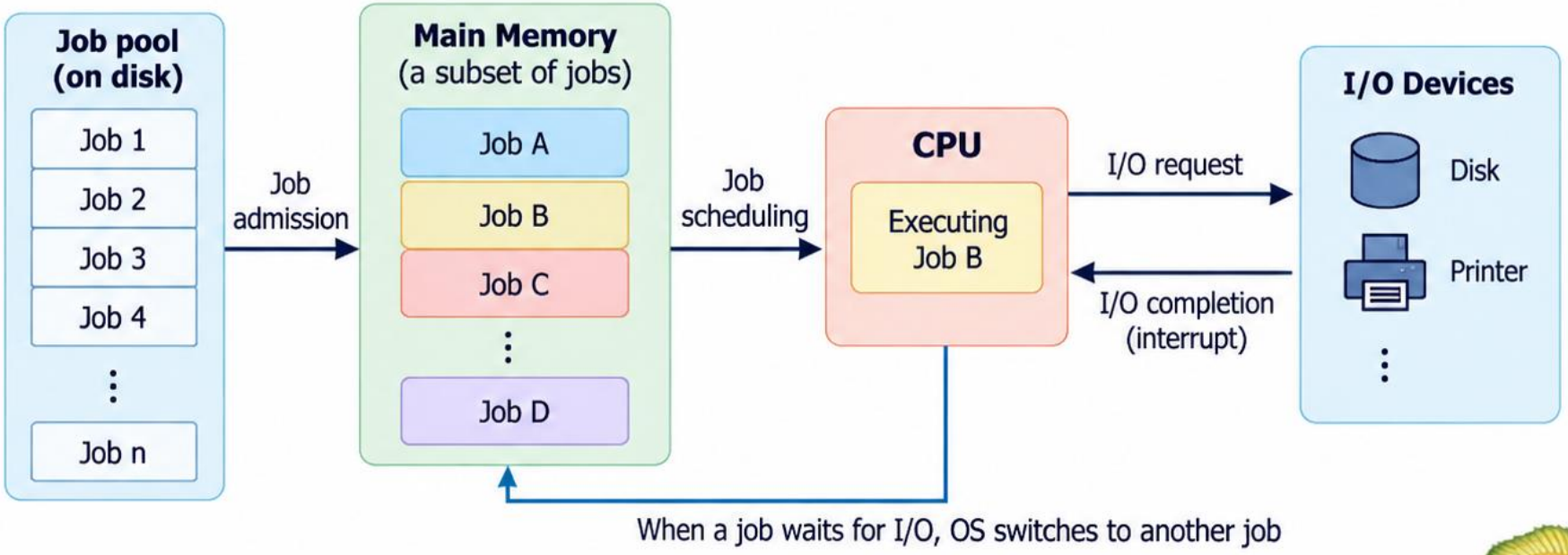
- The bootstrap program is simple and stored in firmware.
- It initializes hardware and loads the OS kernel.
- System programs (daemons) are started at boot time (e.g., systemd on Linux).
- Once booted, the OS waits for events (usually interrupts).
- Interrupts can be hardware interrupts or traps (exceptions).





Multiprogramming (Batch system)

- Single user cannot always keep CPU and I/O devices busy
- Multiprogramming organizes jobs (code and data) so CPU always has one to execute
- A subset of total jobs in system is kept in memory
- One job selected and run via **job scheduling**
- When job has to wait (for I/O for example), OS switches to another job







Multiprogramming and Multitasking

Multiprogramming

- Keeps **several processes** (programs) in memory at the same time.
- The OS selects a process to run. If the process waits (e.g., for I/O), the OS switches to another process.
- As long as at least one process is ready to execute, the **CPU is never idle**.
- Increases CPU utilization and allows more programs to be in the system simultaneously.

Multitasking

- A logical extension of multiprogramming.
- The CPU switches among multiple processes frequently, providing a **fast response time** for users.
- Especially important for interactive I/O (e.g., keyboard, mouse, touch screen), which operates at "people speeds."
- Gives the illusion that multiple programs run simultaneously.

Memory Layout for a Multiprogramming System

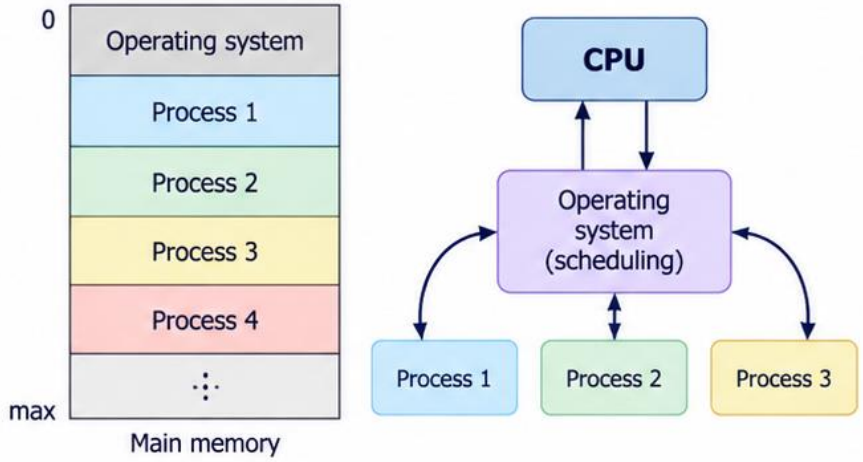


Figure 1.12 Memory layout for a multiprogramming system.

Key Points

- Multiprogramming improves CPU and I/O device utilization by keeping multiple processes in memory.
- Multitasking rapidly switches the CPU among processes to provide fast user response time.
- Requires memory management, CPU scheduling, protection, synchronization, and a file system.
- Virtual memory allows execution of programs larger than physical memory.
- Multiple processes must be isolated to prevent interference and ensure correct operation.





Dual-Mode and Multimode Operation

Dual-Mode Operation

- Computer system must distinguish between operating-system code and **user**-defined code.
- Hardware provides at least two **modes** of operation: **user mode** and **kernel mode** (supervisor/system/ **privileged** mode).
- A mode **bit** in the hardware indicates the current mode: 0 = kernel mode, 1 = user mode.
- Helps protect the OS from errant (or malicious) users and users from one another.

Mode Transitions

- When a user program requests an OS service (via a **system call**), the system transitions from user mode to kernel mode.
- A **trap or interrupt** also causes a switch to kernel mode.
- The OS always switches back to user mode before passing control to a user program.
- At system boot time, the hardware starts in kernel mode, loads the OS, and then starts user applications in user mode.
- The instruction to switch to kernel mode is a **privileged instruction**.

Privileged Instructions

- Some instructions can cause harm (e.g., I/O control, timer management, interrupt management) and are executed only in kernel mode.
- If a user program attempts to execute a privileged instruction in user mode, the hardware treats it as illegal and traps to the OS.

Transition from User to Kernel Mode

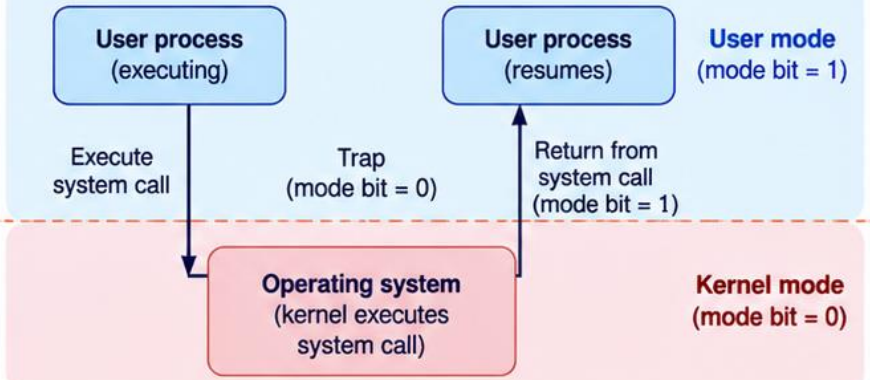


Figure 1.13 Transition from user to kernel mode.

Multimode Operation

- Some architectures provide more than two modes.
- Example: Intel processors have four protection rings (ring 0 = kernel, ring 3 = user; rings 1 and 2 rarely used).
- ARMv8 systems have seven modes.
- CPUs with virtualization often have an additional mode for the virtual machine manager (VMM), with more privileges than user processes but fewer than the kernel.

Key Takeaways

- Dual-mode (and multimode) operation provides protection, security, and stability.
- System calls are the controlled way for user programs to request OS services.
- Hardware traps violations (e.g., illegal instructions or memory access) and transfers control to the OS, which handles the error (often by terminating the program).





Timer

- Timer to prevent infinite loop (or process hogging resources)
  - Timer is set to interrupt the computer after some time period
  - Keep a counter that is decremented by the physical clock
  - Operating system set the counter (privileged instruction)
  - When counter zero generate an interrupt
  - Set up before scheduling process to regain control or terminate program that exceeds allotted time





The Timer: Keeping Control of the CPU

A timer ensures that the operating system maintains control over the CPU by generating an interrupt after a specified time period.

1 OS Sets the Timer

Operating System

Set timer (e.g., 10 ms)

Timer

- Counter = N
- Clock ticks at fixed rate (e.g., 1 ms)
- Counts down

2 Timer Counts Down

Clock (fixed rate)

Tick (e.g., 1 ms)

Timer

Counter = N
(counter decremented every tick)

Example

10-bit counter
1 ms clock

- Interrupts from 1 ms to 1,024 ms
- In steps of 1 ms

3 Timer Interrupt Occurs

When the counter reaches 0, a timer interrupt is generated.

Counter = 0

Interrupt

Operating System

Regain control of the CPU

Ensuring Fair CPU Use

User Program
(runs for a time slice)

```
...
while (true) {
  ...
}
...
```

Timer interrupt
(prevents infinite loops)

Operating System

Handle interrupt (e.g., schedule next process)

Fixed vs. Variable Timer

Fixed Timer

- Interrupt at a fixed interval
- Example: 1/60 second (~16.7 ms)

Variable Timer

- Implemented using a fixed-rate clock and a counter
- OS sets the counter
- Allows a range of interrupt intervals

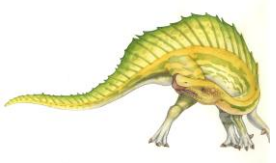
Key Points

- A timer prevents a user program from monopolizing the CPU.
- The OS sets the timer before giving control to a user program.
- The counter is decremented on each clock tick.
- When the counter reaches 0, an interrupt transfers control to the OS.
- Timer interrupts can be used for process scheduling or treated as an error, depending on the situation.





Resource Management



Process Management (1 of 2)

A process is an instance of a program in execution.

1 Program vs Process

Program (Passive)

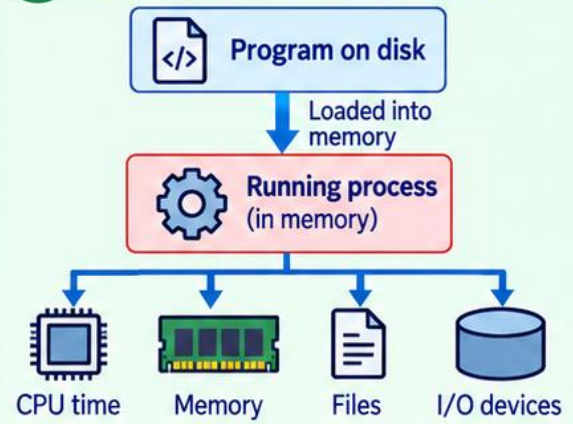
- Stored on disk
- Set of instructions
- Not executing by itself

Load and execute

Process (Active)

- Program in execution
- Uses CPU time
- Uses memory, files, and I/O
- Active entity

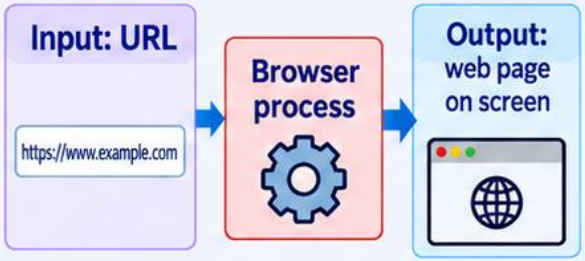
2 How a Process Runs



Resources are allocated while the process runs.

When the process terminates, the OS reclaims reusable resources.

3 Example: Web Browser Process



- Receives input data
- Executes instructions and system calls
- Displays the requested page.

4 Key Ideas

A **program** by itself is **not** a process.

A process is the unit of work in a system.

Processes may create subprocesses that execute concurrently.

One program can have many separate process instances.

Program = passive instructions | Process = active execution



Process Management (2 of 2)

The operating system manages process execution, concurrency, and coordination.

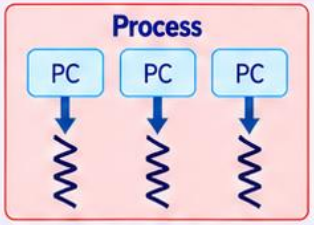
1 Single-Threaded vs. Multithreaded

Single-Threaded Process



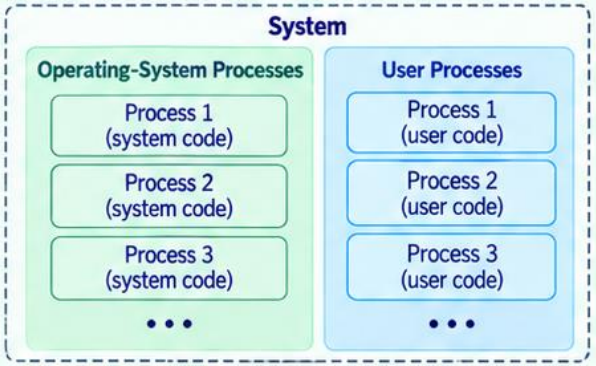
- One program counter
- Sequential execution
- One instruction at a time

Multithreaded Process



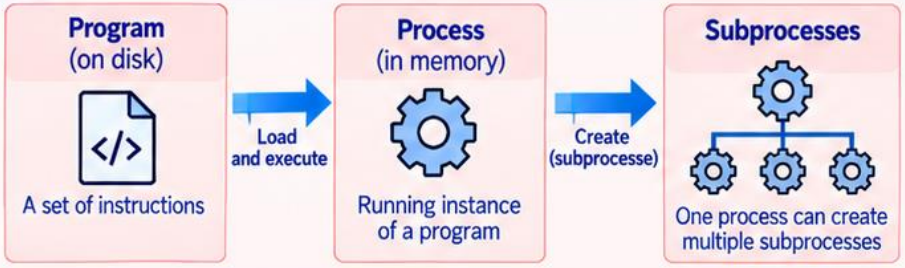
- Multiple threads
- Multiple program counters
- Can perform multiple execution sequences

2 Processes in a System



All processes can execute concurrently.

3 Process Creation and Execution



- System calls can create subprocesses
- Each process needs CPU time, memory, files, and I/O
- A process is the unit of work in a system

4 OS Responsibilities

- Create and delete user and system processes
- Schedule processes and threads on CPUs
- Suspend and resume processes
- Provide process synchronization
- Provide process communication

Process management = creating, scheduling, coordinating, and communicating work in the system.



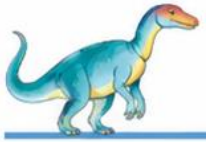


Process Management Activities

The operating system is responsible for the following activities in connection with process management:

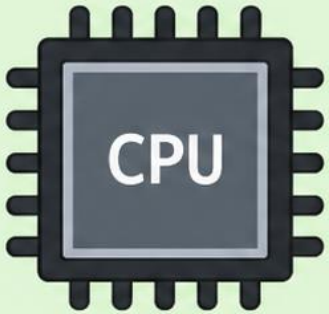
- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling





Memory Management (1 of 2)

CPU



- Fetches instructions from memory
- Reads and writes data in memory

instruction fetch

data read/write

Main Memory

A large byte array

0	10101100
1	01100110
2	00001101
3	11110000
⋮	…
N – 1	01001011

Every byte has an address.

I/O Devices



Share data through main memory.

disk data loaded into memory

Disk (Secondary Storage)



Data on disk must first be loaded into main memory.

Instructions and data must be in main memory before the CPU can use them.



Memory Management (2 of 2)

From Program to Execution (Life Cycle in Memory)



Program on disk
Executable file stored on disk (e.g., .exe)

Mapped to absolute addresses
Loader and linker map the program to memory addresses

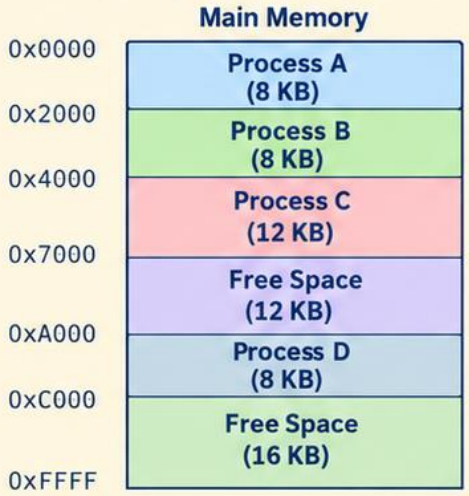
Loaded into memory
Program's code, data, and stack are placed in memory

Process executes
CPU runs the instructions in memory

Process terminates
Process finishes (or is halted)

Memory freed
OS reclaims the memory space for future use

Multiprogramming: Multiple Processes in Memory



Multiple programs can be in memory at the same time (multiprogramming), with each process occupying a separate region.

Operating System Responsibilities

1. Track used/free memory and owner process.
2. Allocate and deallocate memory space.
3. Decide what processes/data move in and out.

Memory management keeps multiple programs in memory safely and efficiently.



Memory Management

- To execute a program all (or part) of the instructions must be in memory
- All (or part) of the data that is needed by the program must be in memory
- Memory management determines what is in memory and when
  - Optimizing CPU utilization and computer response to users
- Memory management activities
  - Keeping track of which parts of memory are currently being used and by whom
  - Deciding which processes (or parts thereof) and data to move into and out of memory
  - Allocating and deallocating memory space as needed



File-System Management (1 of 2)

The operating system gives users a uniform logical view of storage.

Physical Storage Devices



Disk (HDD)



Solid-State Drive (SSD)



Tertiary Storage (tape, optical)

- speed
- capacity
- transfer rate
- access method

OS Abstraction Layer

device details → **Operating System** → uniform file view → File

Converts device-specific details into a logical file abstraction.

Logical Storage Unit: File



programs



text data



binary data



media files

Physical media → OS maps files → Directories organize files → Users access files

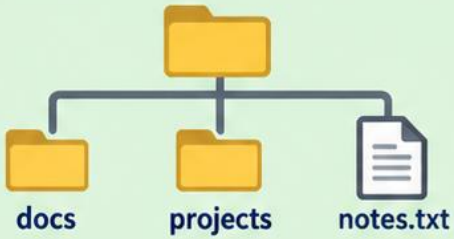
File = related information stored through a logical OS abstraction.



File-System Management (2 of 2)

The OS manages files, directories, storage mapping, protection, and backups.

1 Files & Directories



create / delete files

create / delete directories

2 File Operations



read | write | append | manipulate



Access may depend on permissions:

read, write, append.

3 Mapping to Storage



map logical files onto mass storage

4 Protection & Backup

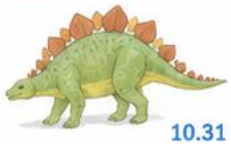


control user access



backup on stable nonvolatile storage

File-system management makes storage convenient, organized, protected, and reliable.





File-system Management

- OS provides uniform, logical view of information storage
  - Abstracts physical properties to logical storage unit - **file**
  - Each medium is controlled by device (i.e., disk drive, tape drive)
    - Varying properties include access speed, capacity, data-transfer rate, access method (sequential or random)
- File-System management
  - Files usually organized into directories
  - Access control on most systems to determine who can access what
  - OS activities include
    - Creating and deleting files and directories
    - Primitives to manipulate files and directories
    - Mapping files onto secondary storage
    - Backup files onto stable (non-volatile) storage media





Mass-Storage Management

The operating system manages secondary and tertiary storage to provide reliable, efficient, and convenient long-term storage of programs and data.

Storage Hierarchy

Secondary Storage (on-line storage)



HDD
(Hard Disk Drive)



NVM / SSD
(Nonvolatile Memory)

- Main on-line storage for programs and data
- Used frequently by the system
- Must be managed efficiently

Tertiary Storage (archival storage)



Magnetic Tape
(drives and tapes)



CD / DVD / Blu-ray
(drives and platters)

- Slower and lower cost (often higher capacity)
- Used for backups, seldom-used data, and long-term archives
- Managed by the OS or by applications

How It Is Used

Programs and data stored on secondary storage (HDD / NVM)

↓

Loaded into main memory for execution and processing

↓

Processing may read from and write to secondary storage devices

OS Responsibilities (Secondary Storage Management)



Mounting and unmounting
Make storage devices available or remove them from the system.



Free-space management
Keep track of available disk space.



Storage allocation
Allocate and deallocate disk space to files and processes.



Disk scheduling
Determine the order of disk I/O requests for efficient access.



Partitioning
Divide storage devices into logical partitions.



Protection
Control access to storage and protect data from unauthorized use.

Efficient management of secondary storage is critical to overall system performance.

The OS may also manage tertiary storage, including mounting/unmounting media, allocating devices, and migrating data between secondary and tertiary storage.

Silberschatz, Galvin and Gagne ©2018



Mass-Storage Management

- Usually disks used to store data that does not fit in main memory or data that must be kept for a “long” period of time
- Proper management is of central importance
- Entire speed of computer operation hinges on disk subsystem and its algorithms
- OS activities
  - Mounting and unmounting
  - Free-space management
  - Storage allocation
  - Disk scheduling
  - Partitioning
  - Protection





Caching

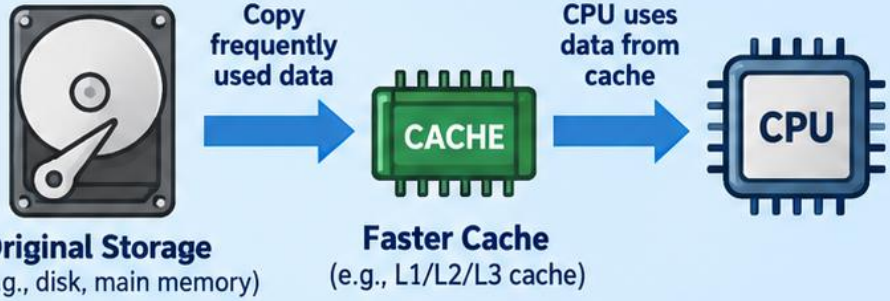
- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
  - If it is, information used directly from the cache (fast)
  - If not, data copied to cache and used there
- Cache smaller than storage being cached
  - Cache management important design problem
  - Cache size and replacement policy



Cache Management (1 of 2)

Caching copies frequently used information into faster storage.

1. How Caching Works



Original Storage
(e.g., disk, main memory)

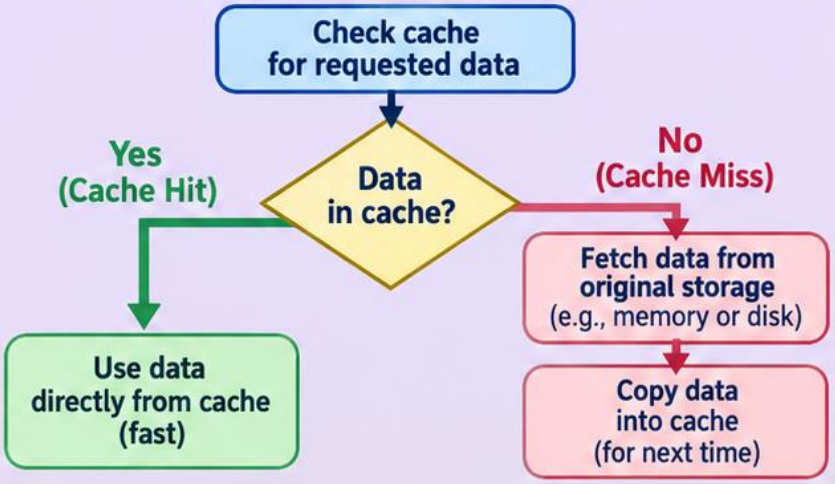
Slower, larger storage

Faster Cache
(e.g., L1/L2/L3 cache)

Smaller, faster storage

Fast processing

2. Cache Hit vs. Cache Miss



3. Why Caches Help



Faster access

Get data quickly



Fewer waits

Spend less time waiting



Better performance

Overall system runs faster

4. Who Manages Each Level

Registers



Managed by programmer / compiler

(explicitly used in code)

Hardware Caches



Managed by hardware

(automatic, transparent to software)

Disk-to-Memory Transfers



Managed by the operating system

(e.g., page cache, virtual memory)

Cache = small, fast temporary copy of data likely to be used again soon.



Characteristics of Various Types of Storage

Level	1	2	3	4	5
Name	registers	cache	main memory	solid-state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25-0.5	0.5-25	80-250	25,000-50,000	5,000,000
Bandwidth (MB/sec)	20,000-100,000	5,000-10,000	1,000-5,000	500	20-150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

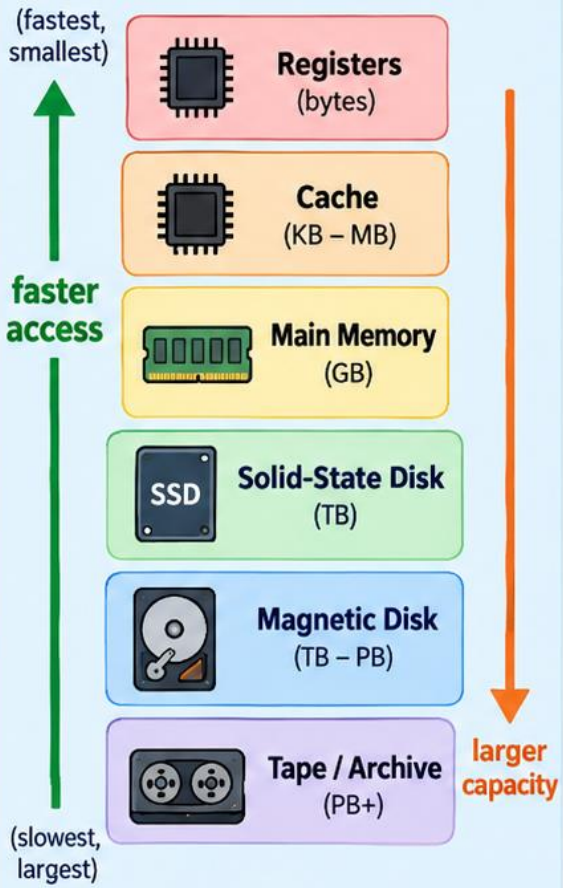
Movement between levels of storage hierarchy can be explicit or implicit



Cache Management (2 of 2)

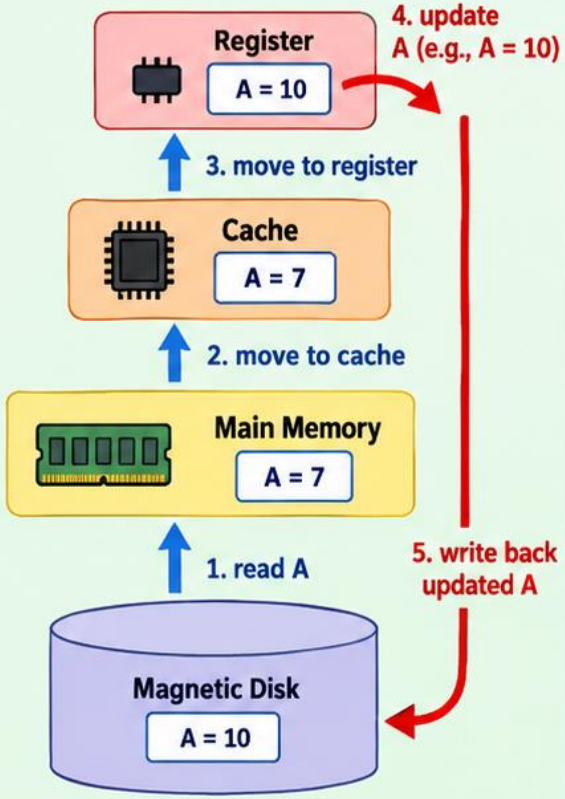
1 Storage Hierarchy

Different levels trade speed for capacity.



2 Data Migration Example: Integer A

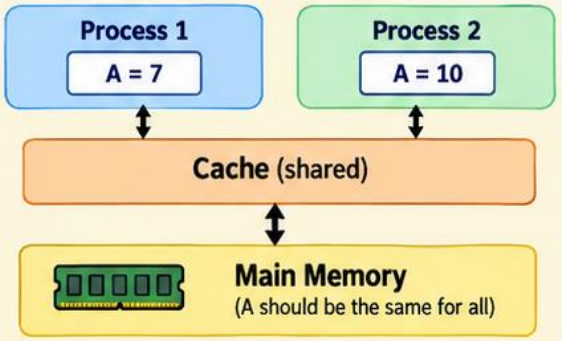
A is loaded from disk up to a register, then updated and written back.



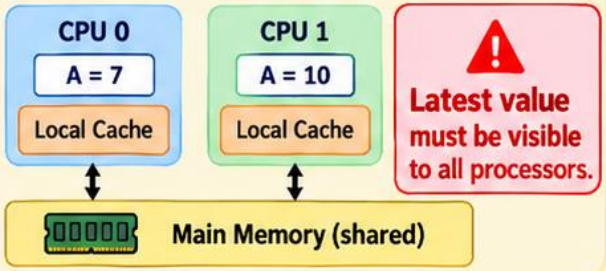
3 Consistency Problems

The same data may exist in multiple places. All copies must see the latest value.

Multitasking (same machine)



Multiprocessor (local caches)



- **Cache coherency** is a hardware-level issue (e.g., coherence protocols).
- **In distributed systems,** replicas make consistency more complex (e.g., networks, failures).

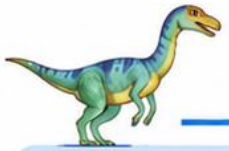
When the same data exists in many places, the system must keep copies consistent.



Migration of data “A” from Disk to Register

- Multitasking environments must be careful to use most recent value, no matter where it is stored in the storage hierarchy
- Multiprocessor environment must provide **cache coherency** in hardware such that all CPUs have the most recent value in their cache
- Distributed environment situation even more complex
  - Several copies of a datum can exist
  - Various solutions covered in Chapter 19





I/O System Management

The operating system hides the peculiarities of specific I/O hardware devices from the user and from most of the operating system itself.

I/O Subsystem Components

1 Memory-Management Component

Includes buffering, caching, and spooling.



Buffering
Temporarily store data in memory



Caching
Keep frequently used data in faster storage



Spooling
Queue I/O data (for example, for printers)

2 General Device-Driver Interface

Provides a uniform interface for device drivers so the rest of the OS can access I/O devices in a consistent way.



3 Device Drivers for Specific Hardware

Drivers for individual devices (e.g., disk, keyboard, mouse, network card, printer).



Disk Drive



Keyboard



Mouse



Network Card



Printer

How I/O System Management Works

User / Applications

↓

Operating System (I/O Subsystem)
Provides uniform access to I/O devices

- Buffering Caching Spooling
- Device-Driver Interface
- Device Drivers (for specific devices)

↓

Device Drivers
Handle device-specific details

↓

Hardware Devices
(disk, keyboard, mouse, network, printer, etc.)

Only the device driver knows the peculiarities of the specific device to which it is assigned.

Silberschatz, Galvin and Gagne ©2018



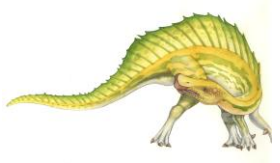
I/O Subsystem

- One purpose of OS is to hide peculiarities of hardware devices from the user
- I/O subsystem responsible for
  - Memory management of I/O including buffering (storing data temporarily while it is being transferred), caching (storing parts of data in faster storage for performance), spooling (the overlapping of output of one job with input of other jobs)
  - General device-driver interface
  - Drivers for specific hardware devices





Protection and security





Protection and Security

- **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS
- **Security** – defense of the system against internal and external attacks
  - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
- Systems generally first distinguish among users, to determine who can do what
  - User identities (**user IDs**, security IDs) include name and associated number, one per user
  - User ID then associated with all files, processes of that user to determine access control
  - Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file
  - **Privilege escalation** allows user to change to effective ID with more rights





Security and Protection

In a multiuser, multiprogramming system, access to data and resources must be regulated so that only authorized users and processes can use them.

Protection

Protection is any mechanism for controlling the access of processes or users to the resources of a computer system.

- **Controls access to resources**
  Ensures that files, memory segments, CPU, and other resources can be used only by authorized processes.



- **Hardware enforcement**
  Examples: memory-addressing hardware restricts access to a process's own address space; a timer prevents a process from monopolizing the CPU; device-control registers are not accessible to users.



- **Specify and enforce controls**
  A protection mechanism must provide ways to specify the controls and to enforce them.



- **Improves reliability**
  Helps detect interface errors early, preventing one faulty subsystem from affecting others.



Privilege Escalation



- Sometimes a user needs extra permissions (for example, to access a restricted device).
- Operating systems provide mechanisms for privilege escalation.
- **Example (UNIX):** A program with the setuid attribute runs with the owner's user ID (effective UID) instead of the current user's ID, until it turns off the extra privileges or terminates.

Security

Security defends the system from external and internal attacks.

- **Protects against threats**
  Includes viruses and worms, denial-of-service attacks (which use system resources), identity theft, and theft of service (unauthorized use).



- **Operating system role**
  Some operating systems provide security features; others leave them to policy or additional software.



- **Growing importance**
  Due to the rise in security incidents, OS security features are a fast-growing area of research and implementation.



Users, User IDs, and Groups

- **Each user has a unique user identifier (user ID)** (called a security ID (SID) in Windows).
- **Authentication**
  When a user logs in, the system determines the user's ID, which is associated with all of the user's processes and threads.
- **Groups**
  Users can be members of one or more groups, each with a group name and group identifier. Group permissions allow different access rights (e.g., owner can read/write, group members can read).



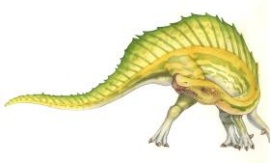


Together, protection and security ensure safe, reliable, and controlled use of system resources.

Silberschatz, Galvin and Gagne ©2018



Virtualization

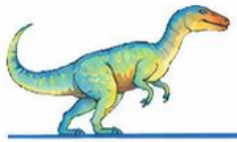




Virtualization

- Allows operating systems to run applications within other OSes
  - Vast and growing industry
- **Emulation** used when source CPU type different from target type (i.e. PowerPC to Intel x86)
  - Generally slowest method
  - When computer language not compiled to native code – **Interpretation**
- **Virtualization** – OS natively compiled for CPU, running **guest** OSes also natively compiled
  - Consider VMware running WinXP guests, each running applications, all on native WinXP **host** OS
  - **VMM** (virtual machine Manager) provides virtualization services





Virtualization (1 of 2)

Virtualization abstracts one physical computer into multiple execution environments.

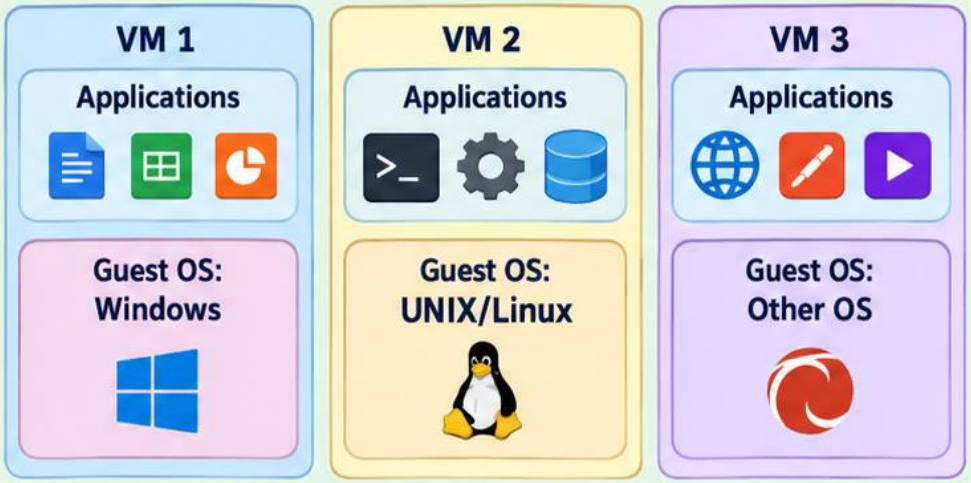
Physical Hardware



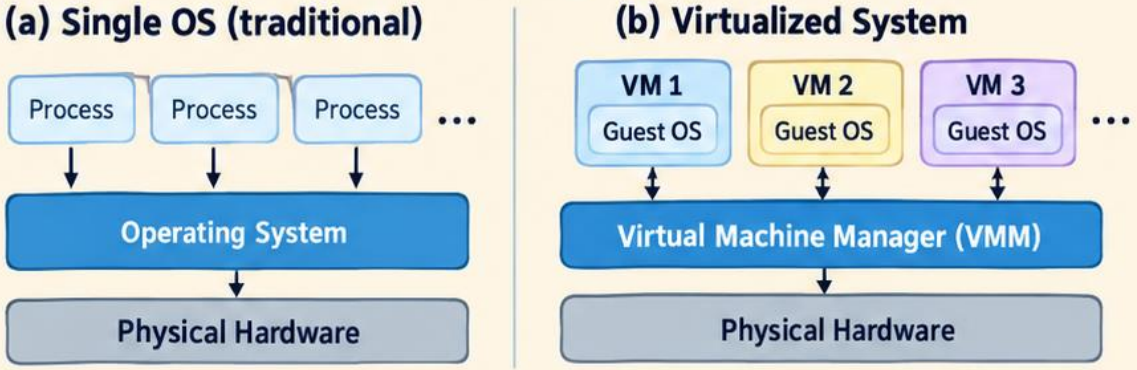
Virtual Machine Manager (VMM)

Physical Hardware

Multiple Virtual Machines



Single OS vs Virtualized System



Key Points

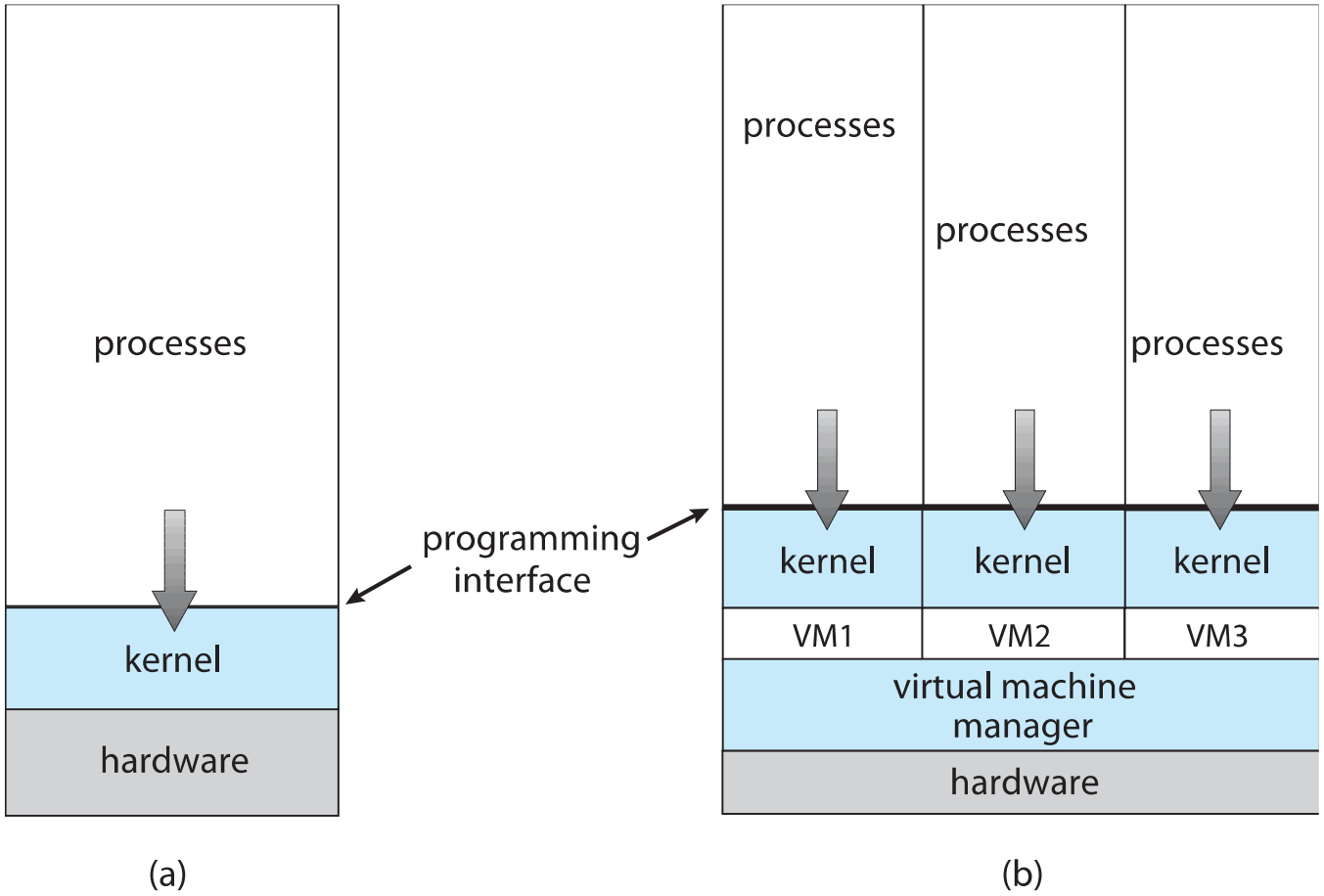
1. Each VM appears to have its own private computer.
2. Users can switch among operating systems.
3. Guest operating systems can run at the same time and interact.

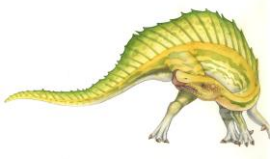


Silberschatz, Galvin and Gagne ©2018



Computing Environments - Virtualization



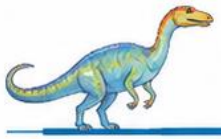




Virtualization (cont.)

- Use cases involve laptops and desktops running multiple OSes for exploration or compatibility
  - Apple laptop running Mac OS X host, Windows as a guest
  - Developing apps for multiple OSes without having multiple systems
  - Quality assurance testing applications without having multiple systems
  - Executing and managing compute environments within data centers
- VMM can run natively, in which case they are also the host
  - There is no general-purpose host then (VMware ESX and Citrix XenServer)



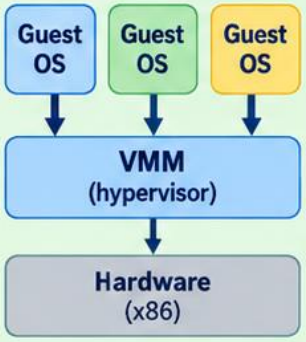


Virtualization (2 of 2)

Virtualization vs. Emulation

Virtualization

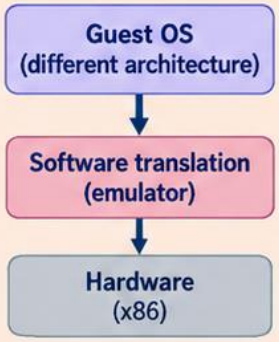
Same CPU architecture (e.g., x86 → x86)



Runs native guest OSs near-native performance.

Emulation

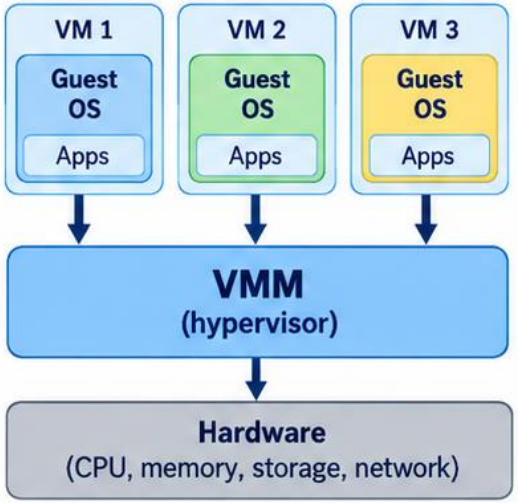
Different CPU architecture (e.g., ARM → x86)



Simulates different hardware slower execution.

Emulation translates machine instructions; virtualization runs native code on compatible hardware.

Role of the VMM



- Runs guest operating systems
- Manages CPU, memory, storage, network
- Protects each guest from others

Where Virtualization Is Used



Laptops/desktops: run another OS
(e.g., Windows, Linux, macOS on the same machine)



Software testing: many OSs on one server
(e.g., different OS versions and configurations)



Data centers: manage computing environments
(e.g., consolidation, cloud infrastructure)

Host-based VMM vs. Bare-metal VMM

Host-based VMM (Type 2)



Runs as an application on a host OS.

Example: desktop virtualization (e.g., personal computers)

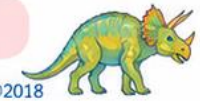
Bare-metal VMM (Type 1)



Runs directly on hardware (no general-purpose host OS).

Example: server/data-center virtualization (e.g., enterprise servers)

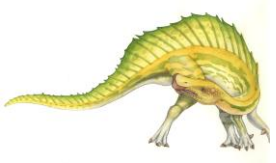
Virtualization improves flexibility by letting one physical machine behave like many independent computers.



Silberschatz, Galvin and Gagne ©2018



Distributed systems

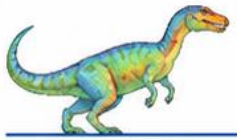




Distributed Systems

- Collection of separate, possibly heterogeneous, systems networked together
  - **Network** is a communications path, **TCP/IP** most common
    - **Local Area Network** (**LAN**)
    - **Wide Area Network** (**WAN**)
    - **Metropolitan Area Network** (**MAN**)
    - **Personal Area Network** (**PAN**)
- **Network Operating System** provides features between systems across network
  - Communication scheme allows systems to exchange messages
  - Illusion of a single system





1.8 Distributed Systems (1 of 2)

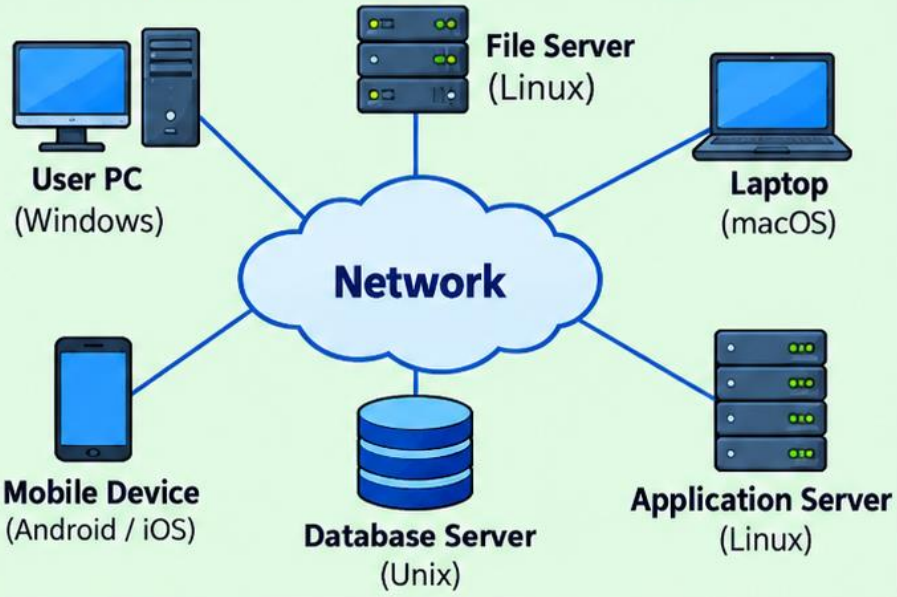
What is a Distributed System?

- A **distributed system** is a collection of physically separate, possibly heterogeneous computer systems that are networked to provide users with access to the various resources that the system maintains.
- Sharing resources through a network increases computation speed, functionality, data availability, and reliability.

Networks

- A **network** is a communication path between two or more systems.
- Distributed systems depend on networking for their functionality.
- The protocols used, distances between nodes, and transport media greatly affect a system's utility and popularity.
- **TCP/IP** is the most common network protocol and provides the fundamental architecture of the Internet. Most general-purpose operating systems support TCP/IP.
- Some systems support proprietary protocols to suit their needs.

Example of a Distributed System



Types of Networks (by distance)

PAN	**Personal-area network** – a few feet (e.g., Bluetooth, 802.11 between a phone and a headset).
LAN	**Local-area network** – within a room, building, or campus.
MAN	**Metropolitan-area network** – within a city.
WAN	**Wide-area network** – links buildings, cities, or countries (e.g., a global company's network).

Silberschatz, Galvin and Gagne ©2018



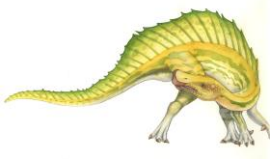
Distributed Systems (2 of 2)

- **Networks** are characterized based on the distance between nodes.
  - **Personal Area Network (PAN)** – a few feet (e.g., Bluetooth, 802.11)
  - **Local Area Network (LAN)** – within a room, building, or campus
  - **Metropolitan Area Network (MAN)** – within a city
  - **Wide Area Network (WAN)** – links buildings, cities, or countries
    - A global company may have a WAN to connect its offices worldwide.
- **Network media** include:
  - Copper wires, fiber strands, wireless transmissions (satellites, microwave dishes, radios), cellular networks, and even short-range infrared.
- **Network Operating System**
  - Provides features such as file sharing across the network.
  - Allows different processes on different computers to exchange messages.
  - Each computer runs autonomously but is aware of the network.
- **Distributed Operating System**
  - Provides a less autonomous (more tightly integrated) environment, giving users the illusion of a single system.





Kernel Data Structure





Lists, Stacks, and Queues

- An **array** allows direct access to each element, but it is not well-suited for items of varying size or for easy insertion and deletion when relative positions must be preserved.
- A **list** represents a collection of data values as a sequence, where items are accessed in a particular order.
- The most common implementation of a list is a **linked list**.

Types of Linked Lists

1. Singly Linked List

- Each item points to its successor.



Figure 1.17 Singly linked list.

2. Doubly Linked List

- Each item can refer to its predecessor or its successor.

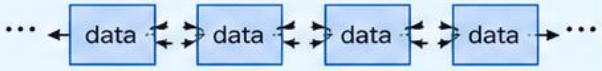


Figure 1.18 Doubly linked list.

3. Circularly Linked List

- The last element refers to the first element, rather than to null.

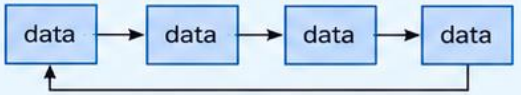


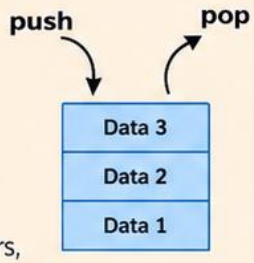
Figure 1.19 Circularly linked list.

Characteristics of Linked Lists

- Accommodate items of **varying sizes**.
- Allow easy insertion and deletion of items.
- Retrieving a specified item in a list of size n is linear — $O(n)$ in the worst case.
- Used by kernel algorithms and as a building block for other data structures.

Stack (LIFO)

- A stack is a sequentially ordered data structure that uses the last in, first out (LIFO) **principle**.
- Operations:
  - **push** – insert an item
  - **pop** – remove an item
- Used in operating systems for function calls (parameters, local variables, and return address are pushed onto the stack).



Queue (FIFO)

- A **queue** is a sequentially ordered data structure that uses the first in, first out (FIFO) principle.
- Items are removed in the order in which they are inserted.
- Everyday examples: shoppers in a checkout line, cars at a traffic signal.

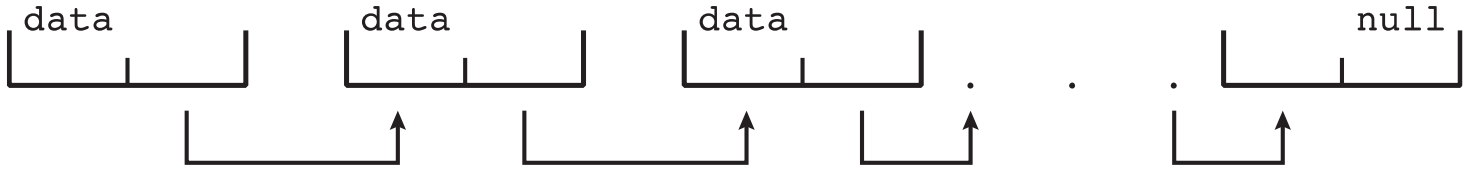




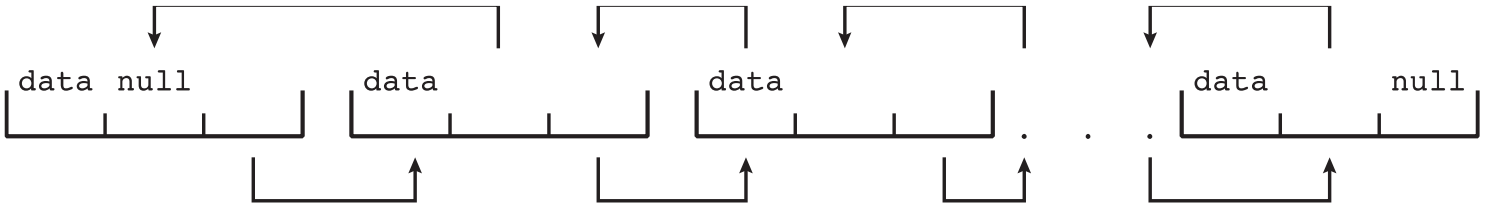


Kernel Data Structures

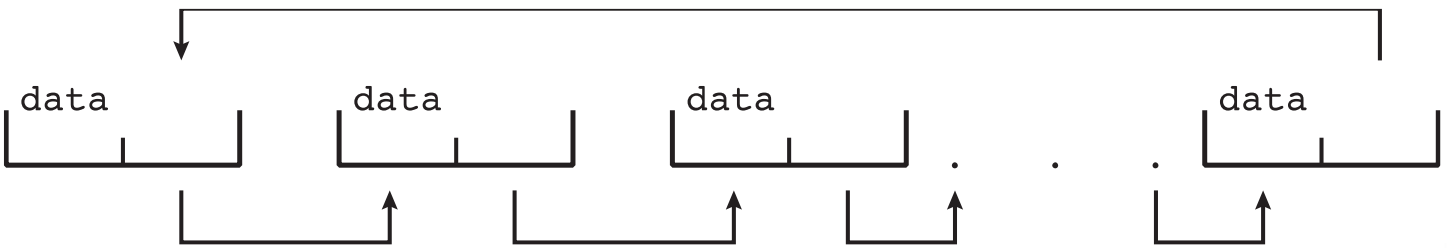
- Many similar to standard programming data structures
- ***Singly linked list***

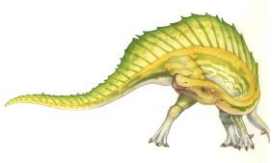


- ***Doubly linked list***



- ***Circular linked list***







Trees

What is a Tree?

- A **tree** is a data structure that can be used to represent data **hierarchically**.
- Data values in a tree are linked through **parent–child relationships**.

Types of Trees

- **General tree**: A parent may have an unlimited number of children.
- **Binary tree**: A parent may have at most two children, called the left child and the right child.
- **Binary search tree (BST)**: A binary tree with an ordering between the parent's two children: **left child ≤ right child**.

Example: Binary Search Tree

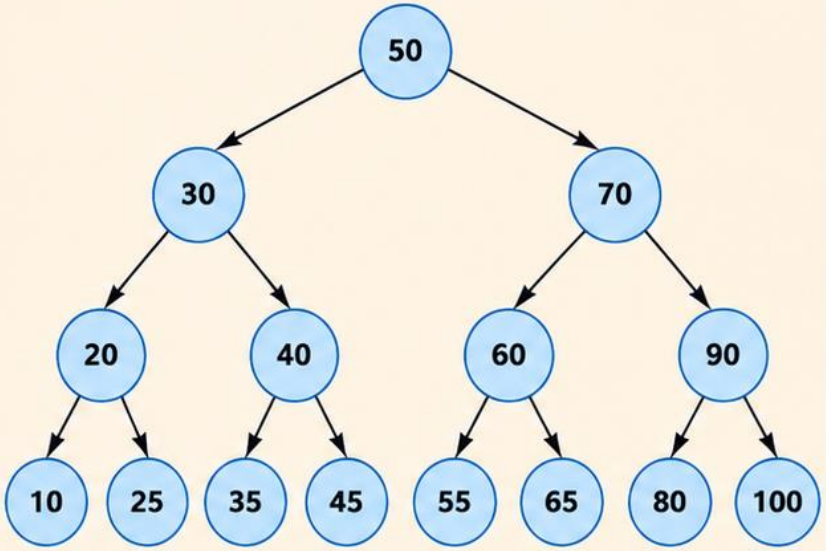


Figure 1.20 Example of a binary search tree.

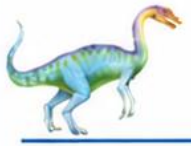
Search Performance

- When we search for an item in a binary search tree, the worst-case performance is $O(n)$ (e.g., when the tree becomes a skewed/linear tree).
- To remedy this, we can use a balanced binary search tree.
- A balanced tree with n items has at most lg n levels, ensuring worst-case performance of $O(\lg n)$.

Use in Operating Systems

- Linux uses a balanced binary search tree (known as a **red-black tree**) as part of its CPU-scheduling algorithm (Section 5.7.1).

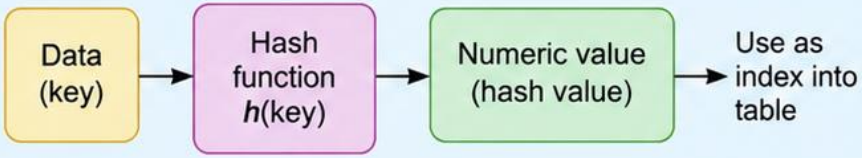




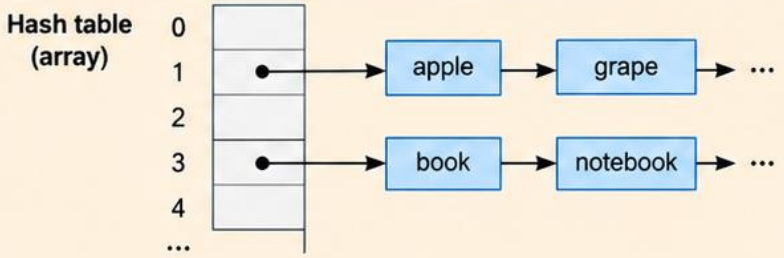
Hash Functions and Maps

- A **hash function** takes data as input, performs a numeric operation on the data, and returns a numeric value.
  - The numeric value (hash value) is used as an index into a table (typically an array) to quickly retrieve the data.
  - Can provide $O(1)$ lookup time on average (versus up to $O(n)$ for a list).
- Hash functions are used extensively in operating systems.
- A potential difficulty is **hash collisions**, where two unique inputs produce the same output value.
  - Collisions can be handled by storing all items with the same hash value in a linked list at that table location.
- A **hash map** associates (maps) **[key : value]** pairs using a hash function.

Hash Function



Handling Collisions (Chaining)



Hash Map

- A **hash map** associates (maps) [key : value] pairs using a hash function.
- To retrieve a value, apply the hash function to the key to find the corresponding table location.
- If multiple keys map to the same location (collision), store all items in a linked list at that location.
- The more collisions, the less efficient the hash function.

Example: User Authentication

1. User enters user name and password.
2. Apply the hash function to the user name to locate the stored password in the hash map.
3. Compare the retrieved password with the password entered by the user.
4. If they match, authentication succeeds.

Hash Map (User Database)

User Name	Password
alice	p@ss1
bob	q7#kL
carol	z9lm2
...	...





Bitmaps

What is a Bitmap?

- A **bitmap** is a string of *n* binary digits that can be used to represent the status of *n* items.
- The value of the ith position in the bitmap is associated with the ith resource.
- Each bit indicates whether a resource is available or unavailable:
  - 0 → resource is available (can also be defined the other way)
  - 1 → resource is unavailable

Example

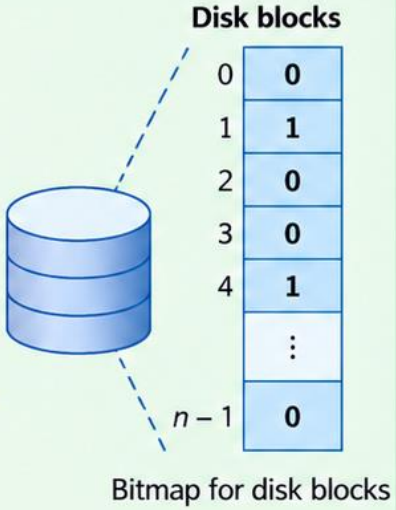
Bitmap:

0	0	1	0	1	1	1	0
0	1	2	3	4	5	6	7

- Resources 2, 4, 5, 6, and 8 are unavailable (bits = 1).
- Resources 0, 1, 3, and 7 are available (bits = 0).

Example: Disk Blocks

- A disk drive can be divided into many individual units called **disk blocks**.
- A bitmap can be used to indicate the availability of each disk block.
- A medium-sized disk might have several thousand disk blocks, and a bitmap provides a space-efficient way to track their usage.



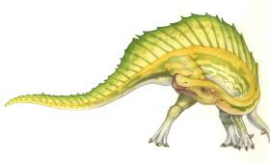
Advantages

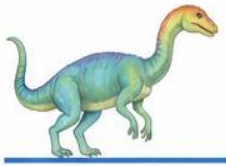
- Bitmaps are very **space-efficient**.
  - Using a single bit per item is 8 times smaller than using an eight-bit Boolean value.
- Commonly used when there is a need to represent the availability of a large number of resources.
- Widely used in operating systems (e.g., for disk block allocation, resource management, etc.).





Computer System Environments





Computing Environments

Operating systems are designed for many hardware and usage contexts.



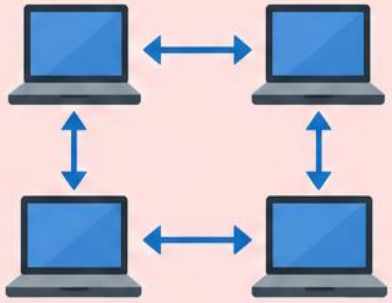
Traditional



Mobile



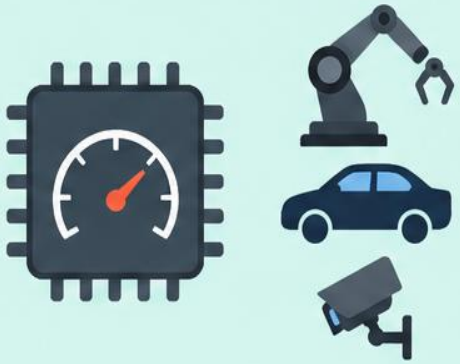
Client–Server



Peer-to-Peer

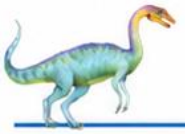


Cloud Computing



Real-Time Embedded



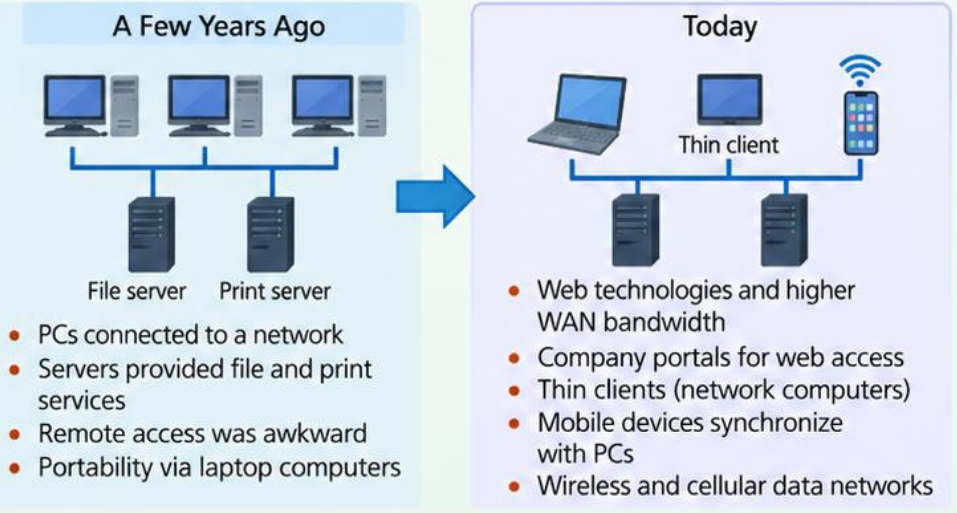


1.10.1 Traditional Computing

Overview

- Traditional computing has evolved, and the lines between environments have blurred.
- Modern environments use web technologies, higher WAN bandwidth, and mobile devices.
- Companies provide portals for web access to internal servers.
- Thin clients (network computers) are used where more security or easier maintenance is desired.

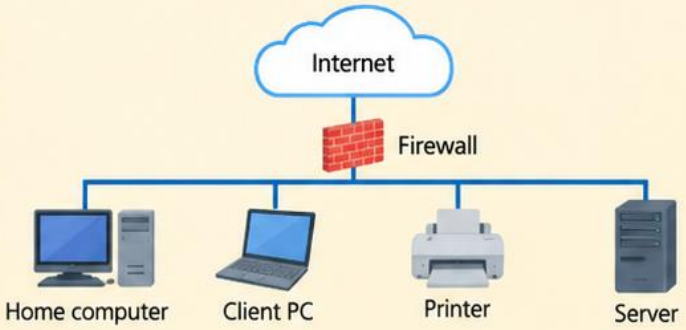
Typical Office Environment (Then and Now)



- PCs connected to a network
- Servers provided file and print services
- Remote access was awkward
- Portability via laptop computers

- Web technologies and higher WAN bandwidth
- Company portals for web access
- Thin clients (network computers)
- Mobile devices synchronize with PCs
- Wireless and cellular data networks

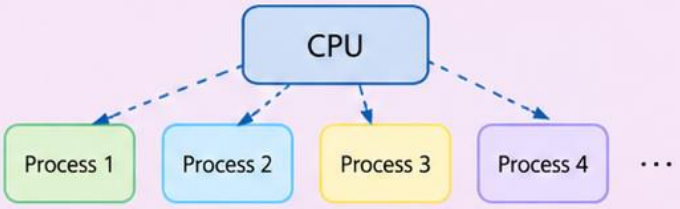
Home Computing Environment



- Faster and more affordable network connections.
- Home computers can serve web pages and run networks.
- Many homes use firewalls to protect against security breaches.
- Firewalls limit communications between devices on a network.

Time-Sharing and Modern Systems

- Historically, systems were either batch or interactive.
- Time-sharing systems used a timer and scheduling algorithms to give multiple users a share of the CPU.
- Traditional time-sharing systems are rare today.
- The same scheduling technique is used in desktops, laptops, servers, and mobile devices.
- Even a single user runs multiple processes (e.g., multiple windows or browser processes), which share the CPU.



Each process gets a slice of CPU time via scheduling.





Mobile

Mobile computing refers to handheld devices such as smartphones, tablets, and other portable devices that provide computing and network connectivity on the go.

Examples



Smartphone



Tablet



Wearable

Functional Difference from a Traditional Laptop

- Designed for high mobility with smaller form factor, touch interfaces, and longer battery life.
- Often run mobile operating systems (e.g., iOS, Android) with app-centric environments.
- Include built-in sensors and hardware not typically found on traditional laptops.

Additional OS Features

- Provide more OS features such as:
  - **GPS** (location services)
  - Gyroscope and accelerometer (motion sensing)
  - Camera, microphone, proximity sensors, etc.

New Types of Applications

- Enable new types of applications, such as:
  - **Augmented reality**
  - Location-based services
  - Mobile gaming
  - Health and fitness apps
  - Social and communication apps

Connectivity

- Use IEEE 802.11 wireless (Wi-Fi) or cellular data networks (e.g., 4G/5G) for connectivity.



Wi-Fi (IEEE 802.11)

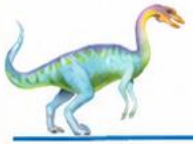


Cellular (4G / 5G)

Major Platforms

- The leading mobile operating systems are **Apple iOS** and **Google Android.**





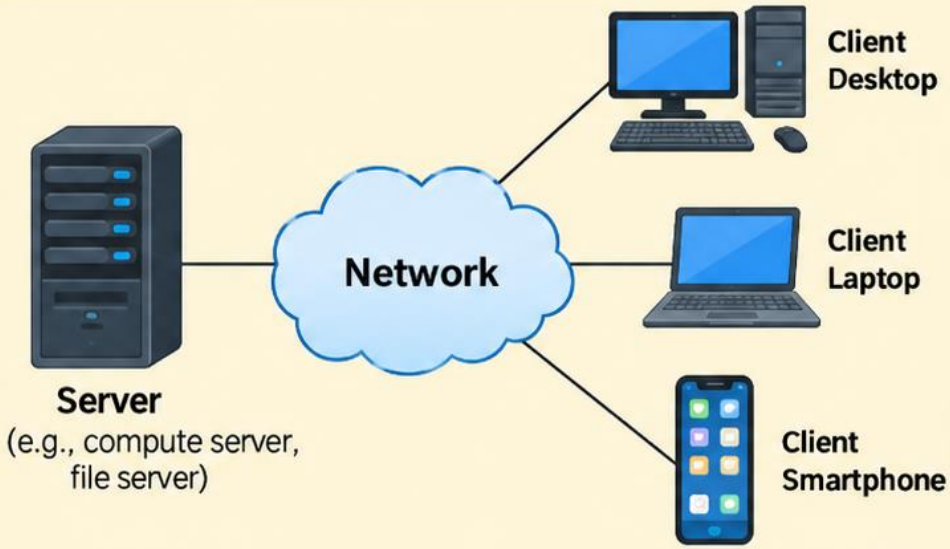
Client–Server

Client–server computing is a distributed computing model in which clients send requests for services, and servers respond to those requests.

Key Points

- Early systems used dumb terminals supplanted by smart PCs.
- Many systems now run as **servers**, responding to requests generated by **clients**.
- A **compute-server system** provides an interface for clients to request services (e.g., database).
- A **file-server system** provides an interface for clients to store and retrieve files.

Client–Server Architecture



Examples of Servers

- **Database server** – handles database requests.
- **File server** – stores and retrieves files.
- **Web server** – delivers web content.
- **Application server** – provides application services.

Examples of Clients

- Desktop computers
- Laptops
- Smartphones and tablets
- Thin clients (network computers)
- Web browsers (as clients)





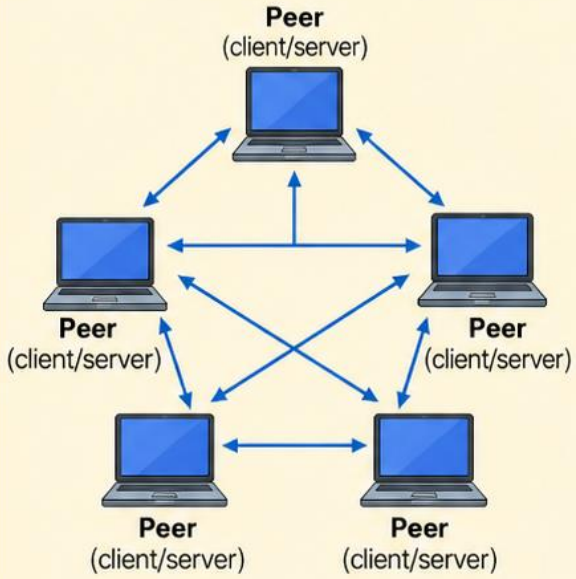
Peer-to-Peer

Peer-to-peer (P2P) is a distributed computing model in which all nodes are equal and can act as both clients and servers.

Key Points

- Another model of distributed system.
- P2P does not distinguish clients and servers.
  - Instead, all nodes are considered peers.
  - Each node may act as a client, a server, or both.
- A node must join the P2P network.
  - Registers its service with a central lookup service on the network, or
  - Broadcasts a request for a service and responds to requests for service via a ***discovery protocol***.
- Examples include Napster, Gnutella, and **Voice over IP (VoIP)** such as Skype.

P2P Network Example



- All nodes are peers.
- Each peer can request and provide services to other peers.
- There is no dedicated central server (though a lookup service may be used).

Examples of P2P Applications

- File sharing systems: Napster, Gnutella
- **Voice over IP (VoIP)**: Skype
- Other examples: BitTorrent, decentralized storage, blockchain networks, etc.



Napster
(file sharing)



Gnutella
(file sharing)



Skype
(VoIP)





Cloud Computing (1/2)

- Cloud computing delivers **computing, storage, and applications** as services across a network, usually the Internet.
- It **extends virtualization** by pooling computing resources and making them available on demand over a network.
- **Amazon EC2** is an example of a cloud computing service that provides many servers, virtual machines, and storage.
- Users **pay based on the resources they use.**



Types of Cloud Computing

Public cloud

Computing resources are owned and operated by a third-party provider and available to the public over the Internet.

Private cloud

Computing resources are dedicated to a single organization, providing greater control, security, and customization.

Hybrid cloud

A combination of public and private clouds, allowing data and applications to move between them.

Software as a Service (SaaS)

Complete applications delivered over the Internet (e.g., email, office suites, collaboration tools).

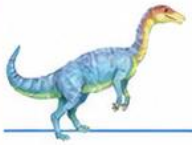
Platform as a Service (PaaS)

A platform with operating systems, databases, and development tools provided as a service for building and deploying applications.

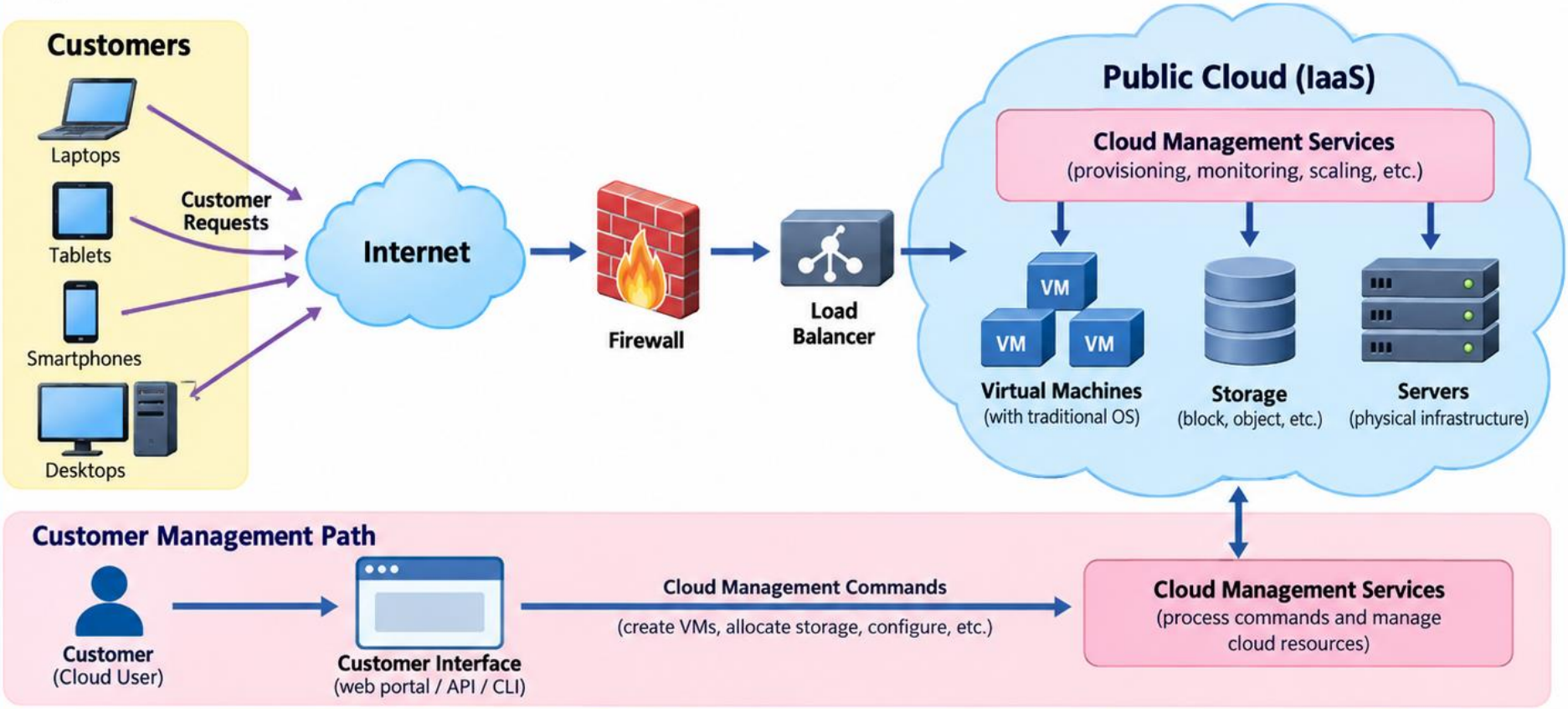
Infrastructure as a Service (IaaS)

Virtual machines, storage, networks, and other infrastructure provided as a service.





Cloud Computing (2/2)



Additional Notes

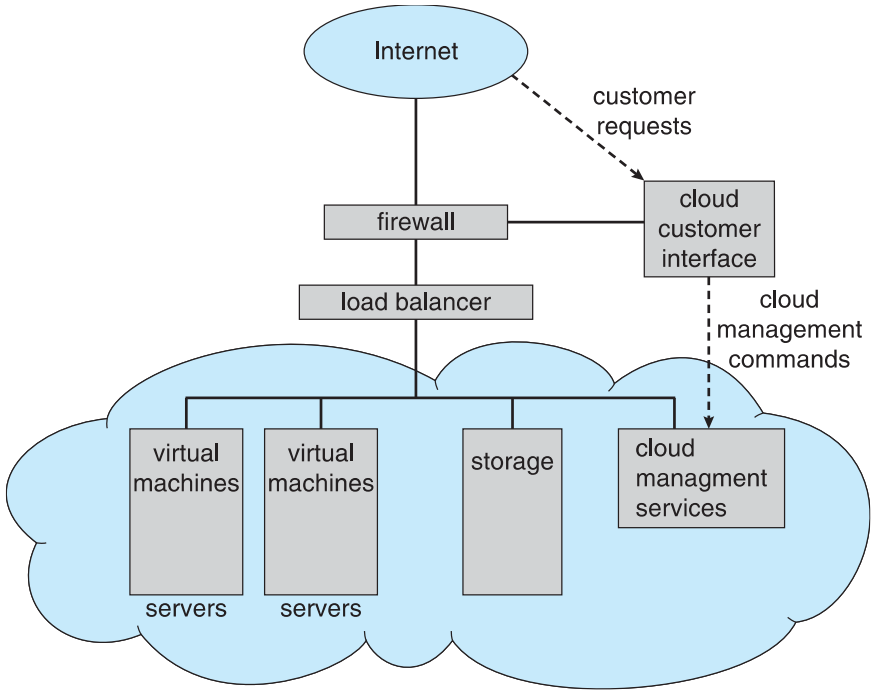
1. **Cloud types can be combined.** Public, private, and hybrid clouds can be used together in a single solution.
2. **Traditional operating systems run inside cloud infrastructure.** e.g., Windows, Linux, and other OSs run in virtual macther OSs run in virtual machines in the cloud.
3. **VMMs manage virtual machines.** A virtual machine monitor (VMM) such as hypervisors (e.g., VMware, KVM, Xen, Hyper-V) creates, runs, and manages virtual machines.
4. **Cloud management tools manage the overall cloud resources and interfaces.** They handle provisioning, monitoring scaling, accounting, and provide user interfaces (web portals, APIs, CLI).
5. **Cloud services and the cloud user interface are typically protected by a firewall.** A firewall helps prevent unauthorized access to cloud services and management interfaces.

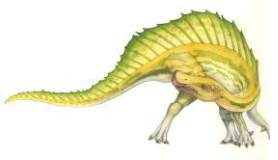




Cloud Computing (cont.)

- Cloud computing environments composed of traditional OSes, plus VMMs, plus cloud management tools
  - Internet connectivity requires security like firewalls
  - Load balancers spread traffic across multiple applications





1.10.6 Real-Time Embedded Systems

Special-purpose computers that monitor and control hardware in the real world.

Often little or no user interface.

Where they are used

Embedded computers are the most common computers and are found everywhere.



Car engine
(engine control)



Manufacturing robot arm
(automation)



Optical drive
(media control)



Microwave oven
(appliance control)



Medical / scientific device
(monitoring)



Smart home
(connected devices)

Use is expanding into smart homes and network-connected devices.

System Variants

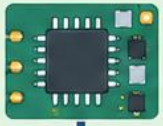
A. General-purpose computer + OS + app



Special-purpose application

General-purpose computer + standard OS (e.g., Linux) + special-purpose app.

B. Special-purpose device + embedded OS



Embedded OS

Special-purpose device with an embedded OS.

C. ASIC / dedicated hardware



No OS

Dedicated hardware that performs specific tasks.

Real-Time Control Loop

Sensors



Measure the environment

Processor / RTOS



Process data and make decisions

Control action / actuators



Affect the physical world



Must finish before deadline.

A real-time system is correct only if it returns the correct result on time.



Free and Open-Source Operating Systems

- Operating systems made available in source-code format rather than just binary **closed-source** and **proprietary**
- Counter to the **copy protection** and **Digital Rights Management** (**DRM**) movement
- Started by **Free Software Foundation** (**FSF**), which has "copyleft" **GNU Public License** (**GPL**)
  - Free software and open-source software are two different ideas championed by different groups of people
    - http://gnu.org/philosophy/open-source-misses-the-point.html/
- Examples include **GNU/Linux** and **BSD UNIX** (including core of **Mac OS X**), and many more
- Can use VMM like VMware Player (Free on Windows), Virtualbox (open source and free on many platforms - http://www.virtualbox.com)
  - Use to run guest operating systems for exploration





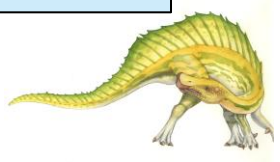
The Study of Operating Systems

There has never been a more interesting time to study operating systems, and it has never been easier. The open-source movement has overtaken operating systems, causing many of them to be made available in both source and binary (executable) format. The list of operating systems available in both formats includes Linux, BUSD UNIX, Solaris, and part of macOS. The availability of source code allows us to study operating systems from the inside out. Questions that we could once answer only by looking at documentation or the behavior of an operating system we can now answer by examining the code itself.

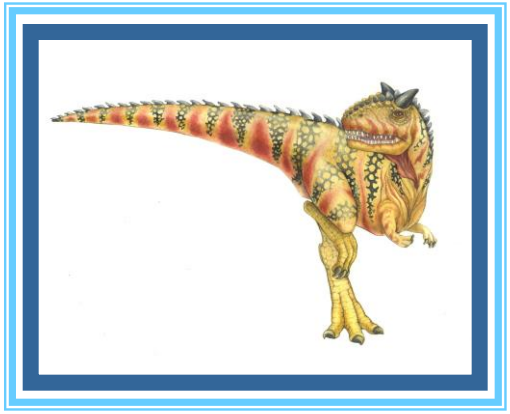
Operating systems that are no longer commercially viable have been open-sourced as well, enabling us to study how systems operated in a time of fewer CPU, memory, and storage resources. An extensive but incomplete list of open-source operating-system projects is available from https://curlie.org/Computers/Software/Operating_Systems/Open_Source/

In addition, the rise of virtualization as a mainstream (and frequently free) computer function makes it possible to run many operating systems on top of one core system. For example, VMware (http://www.vmware.com) providesa free "player" for Windows on which hundreds of free "virtual appliances" can run. Virtualbox (http://www.virtualbox.com) provides a free, open-source virtual machine manager on many operating systems. Using such tools, students can try out hundreds of operating systems without dedicated hardware.

The advent of open-source operating systems has also made it easier to make the move from student to operating-system developer. With some knowledge, some effort, and an Internet connection, a student can even create a new operating-system distribution. Just a few years ago, it was difficult or impossible to get access to source code. Now, such access is limited only by how much interest, time, and disk space a student has.



End of Chapter 1



Silberschatz, Galvin and Gagne ©2018